

**Escuela Técnica Superior de Ingeniería
Universidad de Huelva**

Grado en Ingeniería Informática

Trabajo Fin de Grado

FitEnerGym: Aplicación web para la
gestión de una cadena de gimnasios

Paloma Gómez Quintero
06/09/2023

Resumen

Este proyecto consta de una aplicación web cuyo propósito es adaptar a las nuevas tecnologías el control de la gestión de una cadena de gimnasios. La página permite al personal dado de alta, si su rol es de administrador de gimnasio, registrar y consultar toda la información relacionada con los diferentes trabajadores, socios y gimnasios que están bajo su correspondencia, desde sus datos personales, a sus tablas de ejercicios o pagos. Los trabajadores también podrán acceder a la página, pudiendo acceder a toda su información particular y editar sus datos personales.

El back-end de la aplicación web está desarrollado con el framework Laravel y el front-end con HTML y CSS. El servidor web utilizado es Hostinger y el servidor de base de datos MySQL.

Abstract

This project consists of a web application whose purpose is to adapt the management control of a chain of gyms to new technologies. The page allows registered personnel, if their role is gym administrator, to register and consult all the information related to the different workers, partners and gyms that are under their correspondence, from their personal data, to their exercise tables or Payments. Workers will also be able to access the page, being able to access all their private information and edit their personal data.

The back-end of the web application is developed with the Laravel framework and the front-end with HTML and CSS. The web server used is Hostinger and the database server is MySQL.

Índice General

Resumen	2
Abstract.....	3
Índice General.....	4
Índice de ilustraciones	7
Índice de tablas	8
1. Introducción.....	9
2. Soluciones comerciales.....	10
3. Objetivos.....	12
4. Roles de Usuarios	13
4.1. Rol de Administrador	13
4.2. Rol de Jefe	13
4.3. Rol de Monitor	13
4.4. Rol de Recepcionista	14
4.5. Rol de Socio.....	14
5. Análisis de requisitos.....	14
5.1. Requisitos funcionales	14
5.1.1. Inicio de sesión / Cerrar sesión	15
5.1.2. Gestión de Gimnasio	15
5.1.3. Gestión de Usuarios	16
5.1.4. Gestión de Jefe	17
5.1.5. Gestión de Monitor	17
5.1.6. Gestión de Recepcionista	18
5.1.7. Gestión de Socio.....	19
5.1.8. Gestión de Clase.....	19
5.1.9. Gestión de Sala	20
5.1.10. Calendario de Clases.....	21
5.1.11. Gestión de Ejercicios.....	21
5.1.12. Gestión de Categorías de Ejercicios.....	22
5.1.13. Gestión de Horario de Clases	23
5.1.14. Gestión de Tablas de Ejercicios	23
5.1.15. Gestión de Clases Matriculadas.....	24
5.1.16. Gestión de Historial de pago	24
5.1.17. Gestión de Evolución de Ejercicios.....	25
5.1.18. Gestión de Pasarela de pago	25
5.2. Requisitos no funcionales	25
5.2.1. Seguridad.....	26
5.2.2. Mantenibilidad y sistema	26
5.2.3. Interfaz y rendimiento.....	26
5.3. Diagramas de casos de usos	27
6. Diseño.....	35
6.1. Diagrama Entidad/Relación	35
6.2. Descripción del modelo	37

6.2.1.	Tabla 'Usuario'	37
6.2.2.	Tabla 'Roles'	37
6.2.3.	Tabla 'Role_User'	38
6.2.4.	Tabla 'Gimnasio'	38
6.2.5.	Tabla 'Usuario_Gimnasio'	38
6.2.6.	Tabla 'Sala'	38
6.2.7.	Tabla 'Clases'	39
6.2.8.	Tabla 'Clase_Planificada'	39
6.2.9.	Tabla 'Capacidad_Clase'	39
6.2.10.	Tabla 'Ejercicio'	39
6.2.11.	Tabla 'Usuario_Ejercicio'	40
6.2.12.	Tabla 'Categoria_Ejercicio'	40
6.2.13.	Tabla 'Tabla_de_Ejercicios'	40
6.2.14.	Tabla 'Asignación_Rutina_Ejercicio'	40
6.2.15.	Tabla 'Usuario_Pagos'	41
6.2.16.	Tabla 'Evolucion_Ejercicios'	41
6.2.17.	Tabla 'Evolucion_Ejercicios_Datos'	41
7.	Arquitectura de la aplicación.....	42
7.1.	Aplicación web	42
7.2.	Base de datos	45
7.3.	Interfaces de usuario	46
7.3.1.	Responsive Web Design	46
7.4.	Lenguajes de programación y herramientas	47
7.4.1.	MySQL.....	47
7.4.2.	HTML	47
7.4.3.	CSS	48
7.4.4.	JavaScript.....	48
7.4.5.	Laravel	48
7.4.6.	Bootstrap.....	49
7.4.7.	AdminLTE.....	49
7.4.8.	Visual Studio Code.....	49
7.4.9.	Stripe	50
7.4.10	Hostinger	50
7.4.11	LiteSpeed	50
8.	Diseño de Interfaces de usuario	51
8.1.	Pantalla de Inicio.....	51
8.2.	Pantalla de Inserción con selector	52
8.3.	Pantalla de Inserción sin selector	53
8.4.	Pantalla de Eliminación.....	53
8.5.	Pantalla de Actualización con selector	54
8.6.	Pantalla de Actualización sin selector	55
8.7.	Pantalla de Pasarela de pago.....	55
8.8.	Pantalla de Calendario de Clases	56
8.9.	Pantalla de Entrenamiento Diario de Ejercicios	57
8.10.	Pantalla de Evolución de Ejercicios	58

8.11.	Pantalla de Gráfica de Evolución de Ejercicios.....	59
9.	Implementación.....	60
9.1.	Front-end	61
9.2.	Back-end.....	66
9.3.	Routes	71
10.	Planificación	72
11.	Análisis económico del proyecto	73
11.1.	Costes materiales	74
11.2.	Costes técnicos.....	74
11.3.	Costes de recursos humanos.....	75
11.4.	Coste total del proyecto	75
12.	Conclusiones	76
12.1.	Conclusiones sobre el proyecto	76
12.2.	Líneas futuras	77
13.	Referencias.....	78

Índice de ilustraciones

Ilustración 1: Captura de aplicación Mindbody	10
Ilustración 2: Captura de aplicación Zen Planner	11
Ilustración 3: Captura de aplicación Glofox	11
Ilustración 4: Diagrama de casos de uso del rol de Adminitrador (parte 1).....	27
Ilustración 5: Diagrama de casos de uso del rol de Adminitrador (parte 2).....	28
Ilustración 6: Diagrama de casos de uso del rol de Adminitrador (parte 3).....	29
Ilustración 7: Diagrama de casos de uso del rol de Adminitrador (parte 4).....	30
Ilustración 8: Diagrama de casos de uso del rol de Jefe	31
Ilustración 9: Diagrama de casos de uso del rol de Recepcionista	32
Ilustración 10: Diagrama de casos de uso del rol de Monitor	33
Ilustración 11: Diagrama de casos de uso del rol de Socio	34
Ilustración 12: Diagrama Entidad/Relación de la BD	36
Ilustración 13: Ejemplo Diseño web Responsive	47
Ilustración 14: Diseño de Pantalla de Inicio.....	51
Ilustración 15: Diseño de Pantalla de Inserción con selector	52
Ilustración 16: Diseño de Pantalla de Inserción sin selector	53
Ilustración 17: Diseño de Pantalla de Eliminación.....	53
Ilustración 18: Diseño de Pantalla de Actualización con selector	54
Ilustración 19: Diseño de Pantalla de Actualización sin selector.....	55
Ilustración 20: Diseño de Pantalla de Pasarela de pago	55
Ilustración 21:Diseño de Pantalla del Calendario de Clases	56
Ilustración 22:Diseño de Pantalla de Entrenamiento Diario de Ejercicios	57
Ilustración 23: Diseño de Pantalla de Evolución de Ejercicios	58
Ilustración 24: Diseño de Pantalla de Gráfica de Evolución de Ejercicios	59
Ilustración 25: Estructura del Proyecto.....	60
Ilustración 26: Estructura de la carpeta 'public'	61
Ilustración 27: Estructura de la carpeta 'resources'	62
Ilustración 28: Estructura de la carpeta 'views'	62
Ilustración 29: Estructura de la carpeta 'layouts'	62
Ilustración 30: Estructura de la carpeta 'admin'	63
Ilustración 31: Estructura de la carpeta 'auth'	63
Ilustración 32: Ejemplo de la visualización y creación de una datatable	64
Ilustración 33: Ejemplo de una actualización de datos de un datatable	65
Ilustración 34: Estructura de los Controllers	69
Ilustración 35: Captura de las migraciones.....	70
Ilustración 36: Captura de un listado de rutas.....	71
Ilustración 37: Modelo de cascada con reducción de riesgos	72

Índice de tablas

Tabla 1: Requisito funcional Iniciar/Cerrar sesión	15
Tabla 2: Requisito funcional Gestión de Gimnasios.....	16
Tabla 3: Requisito funcional Gestión de Usuarios	16
Tabla 4: Requisito funcional Gestión de Jefes	17
Tabla 5: Requisito funcional Gestión de Monitores	18
Tabla 6: Requisito funcional Gestión de Recepcionistas	18
Tabla 7: Requisito funcional Gestión de Socios	19
Tabla 8: Requisito funcional Gestión de Clases	20
Tabla 9: Requisito funcional Gestión de Salas	20
Tabla 10: Requisito funcional Calendario de Clase.....	21
Tabla 11: Requisito funcional Gestión de Ejercicios	22
Tabla 12: Requisito funcional Gestión de Categorías de Ejercicios	22
Tabla 13: Requisito funcional Gestión de Horario de Clase.....	23
Tabla 14: Requisito funcional Gestión de Tablas de Ejercicios	24
Tabla 15: Requisito funcional Gestión de Clases Matriculadas	24
Tabla 16: Requisito funcional Gestión de Historial de pago.....	24
Tabla 17: Requisito funcional Gestión de Evolución de Ejercicios.....	25
Tabla 18: Requisito funcional Gestión de Pasarela de pago	25
Tabla 19: Requisito no funcional Seguridad	26
Tabla 20: Requisito no funcional Mantenibilidad y Sistema.....	26
Tabla 21: Requisito no funcional interfaz y rendimient.....	26
Tabla 22: Tabla de costes materiales	74
Tabla 23: Tabla de costes técnico	74
Tabla 24: Tabla de costes de recursos humanos	75
Tabla 25: Tabla de coste total del proyecto.....	75

1. Introducción

En la sociedad actual, el cuidado de la salud y el bienestar personal se han convertido en prioridades fundamentales para un gran número de personas. La conciencia sobre la importancia de llevar un estilo de vida activo y saludable ha generado un creciente interés en la práctica de actividades físicas y el acceso a instalaciones deportivas de calidad.

En este contexto, la gestión eficiente de una cadena de gimnasios se ha convertido en un desafío clave para aquellos que buscan brindar servicios excepcionales a sus clientes y mantener una posición competitiva en el mercado. La evolución de la tecnología ha jugado un papel fundamental en esta área, ya que las aplicaciones web se han convertido en herramientas indispensables para optimizar y simplificar los procesos de gestión de dichos establecimientos.

El presente Trabajo de Fin de Grado (TFG) tiene como objetivo desarrollar una aplicación web de gestión para una cadena de gimnasios, que permita centralizar y automatizar diversas tareas y procesos clave. Esta aplicación ofrecerá a los administradores y empleados una plataforma intuitiva y de fácil acceso para llevar a cabo funciones como la gestión de membresías, el control de asistencia, la planificación de actividades y la gestión financiera, entre otras.

Para llevar a cabo este proyecto, se utilizarán tecnologías de vanguardia y se aplicarán buenas prácticas de desarrollo de software. Además, se llevará a cabo un análisis exhaustivo de los requisitos funcionales y no funcionales para garantizar la satisfacción de las necesidades específicas de la cadena de gimnasios.

Este TFG se estructurará en diferentes etapas, comenzando por la investigación y análisis de los sistemas de gestión existentes en la industria del fitness, así como el estudio de las mejores prácticas y tendencias actuales en el campo de las aplicaciones web. Posteriormente, se procederá al diseño y desarrollo de la aplicación, seguido de pruebas rigurosas para asegurar su correcto funcionamiento y fiabilidad.

En conclusión, este TFG tiene como propósito principal proporcionar una solución efectiva y personalizada para la gestión de una cadena de gimnasios a través de una aplicación web. Se espera que este proyecto contribuya al mejoramiento de la eficiencia operativa de los gimnasios y al ofrecimiento de una experiencia excepcional para sus clientes.

2. Soluciones comerciales

Aquí tienes tres ejemplos reales de aplicaciones web utilizadas para la gestión de cadenas de gimnasios:

- **Mindbody:** Mindbody es una popular plataforma de gestión de gimnasios que ofrece una aplicación web para administrar todas las operaciones del gimnasio. Permite la gestión de horarios de clases, reservas de clientes, pagos y facturación, seguimiento de asistencia, marketing y comunicación con los clientes, informes de rendimiento y más. Además, también proporciona una aplicación móvil para que los clientes puedan reservar clases y acceder a información relevante.

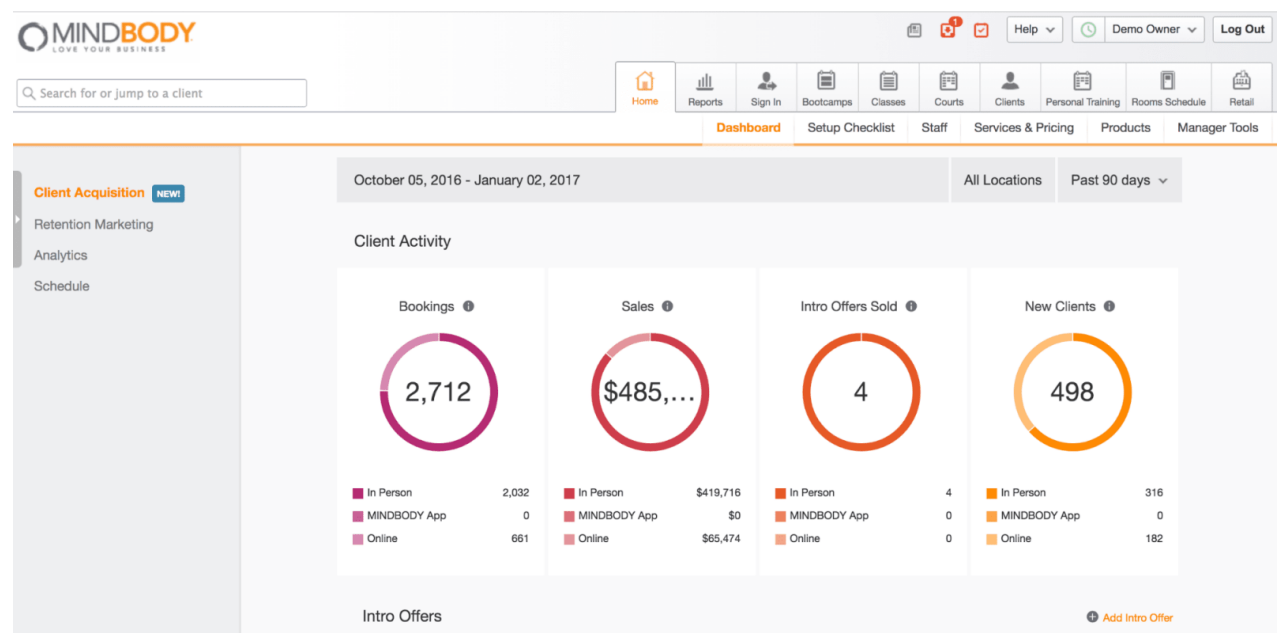


Ilustración 1: Captura de aplicación Mindbody

- **Zen Planner:** Zen Planner es otra solución integral de gestión de gimnasios que ofrece una aplicación web para simplificar todas las funciones operativas. Permite la programación de clases y entrenamientos, gestión de membresías, seguimiento de asistencia, facturación automatizada, integración de pagos, gestión de inventario, marketing y análisis de datos. También proporciona una aplicación móvil para que los clientes reserven clases y realicen pagos.

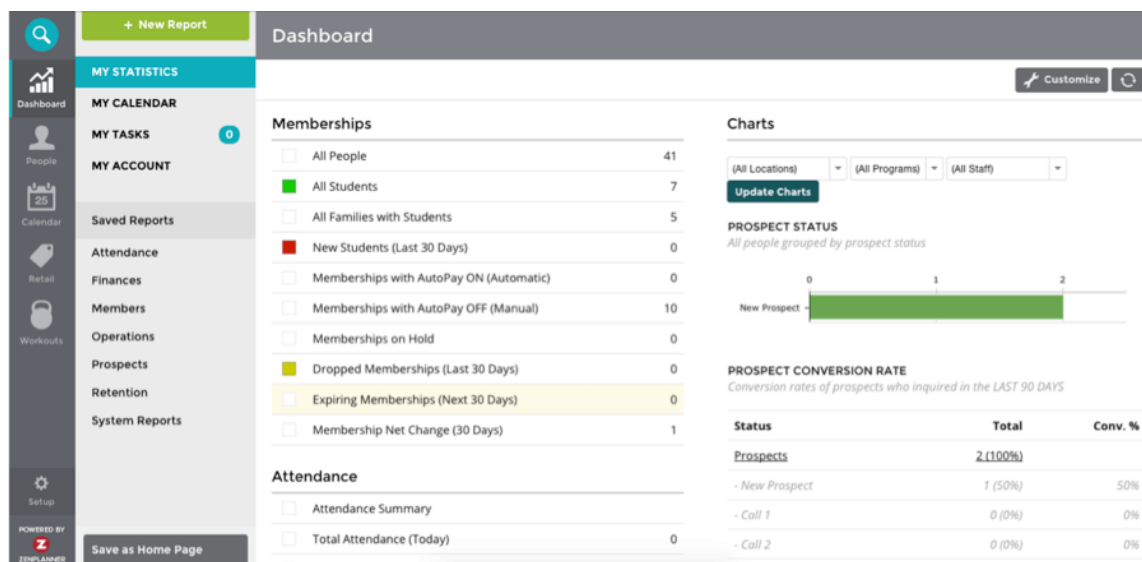


Ilustración 2: Captura de aplicación Zen Planner

- **Glofox:** Glofox es una plataforma de gestión de gimnasios basada en la web que se enfoca en la facilidad de uso y la experiencia del cliente. Ofrece herramientas de reserva y programación de clases, gestión de membresías, pagos y facturación, seguimiento de asistencia, comunicación con los clientes y análisis de datos. Además de la aplicación web, también cuenta con una aplicación móvil para que los clientes reserven clases y realicen pagos de manera conveniente.

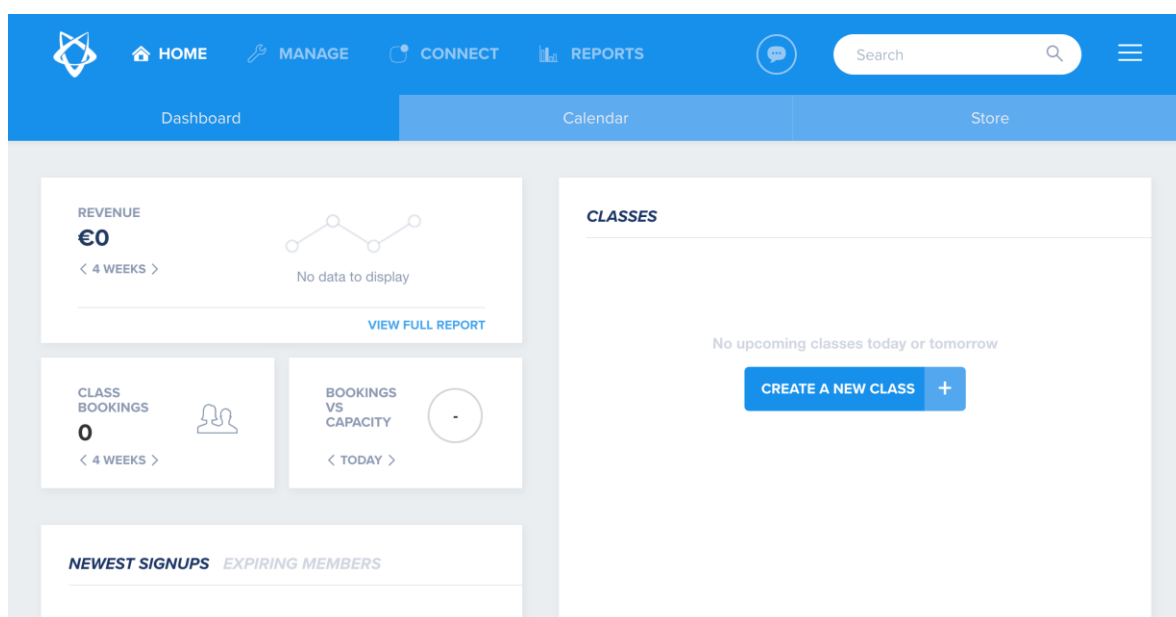


Ilustración 3: Captura de aplicación Glofox

Estas son solo algunas de las muchas soluciones disponibles en el mercado para la gestión de cadenas de gimnasios. Cada una ofrece características específicas, por lo que es importante evaluar las necesidades y requerimientos de tu cadena de gimnasios antes de elegir una aplicación específica.

La diferencia que existe entre estos ejemplos de aplicaciones y la aplicación de este proyecto es la incorporación del seguimiento de la evolución de los socios con sus respectivos ejercicios.

3. Objetivos

Los objetivos para este proyecto sobre una aplicación web de gestión para una cadena de gimnasios serían los siguientes:

- **Analizar y comprender las necesidades y requisitos específicos** de la cadena de gimnasios en términos de gestión y operaciones.
- **Investigar y evaluar los sistemas de gestión existentes** en la industria del fitness, identificando sus fortalezas y debilidades.
- **Diseñar una arquitectura adecuada** para la aplicación web de gestión, teniendo en cuenta la escalabilidad, la seguridad y la usabilidad.
- **Desarrollar la aplicación web de gestión**, implementando las funcionalidades clave requeridas, como la gestión de membresías, el control de asistencia, la planificación de actividades y la gestión financiera.
- **Integrar la aplicación web con otros sistemas** o herramientas utilizadas por la cadena de gimnasios, como sistemas de pago, bases de datos de clientes u otros servicios externos.
- **Realizar pruebas exhaustivas** para asegurar el correcto funcionamiento de la aplicación web y validar su eficiencia en la gestión de la cadena de gimnasios.
- **Evaluar la usabilidad y la experiencia del usuario de la aplicación web**, recopilando comentarios y sugerencias de los administradores y clientes para realizar mejoras si es necesario.
- **Evaluar el impacto de la aplicación web** en la eficiencia operativa de la cadena de gimnasios, analizando indicadores clave como la reducción de tiempo en tareas administrativas, el aumento en la productividad del personal y la mejora en la satisfacción de los clientes.

- **Analizar la seguridad de la aplicación web**, asegurando que los datos de los usuarios y la información confidencial estén protegidos de manera adecuada.
- **Documentar todo el proceso de desarrollo de la aplicación web**, incluyendo los requisitos, el diseño, la implementación, las pruebas y los resultados obtenidos, para que pueda servir como referencia futura y compartir conocimientos sobre el proyecto.

En resumen, los objetivos de este proyecto son analizar, diseñar, desarrollar y evaluar una aplicación web de gestión para una cadena de gimnasios, con el propósito de mejorar la eficiencia operativa y proporcionar una experiencia excepcional tanto para los administradores como para los clientes.

4. Roles de Usuarios

En este apartado se va a enumerar y describirán los diferentes roles de usuario utilizados en esta aplicación:

4.1. Rol de Administrador

El administrador es el encargado principal de la gestión global de la cadena de gimnasios. Tiene acceso a todas las funciones y datos de la aplicación. Puede agregar o eliminar gimnasios, gestionar usuarios, asignar roles, supervisar los pagos de los socios, ver las evoluciones de los socios y tomar decisiones estratégicas para el negocio. Es el nivel más alto de acceso y tiene control total sobre la plataforma.

4.2. Rol de Jefe

El jefe de cada gimnasio es responsable de la administración y operación de su respectivo gimnasio. Tienen acceso a funciones relacionadas con la gestión interna de su local, como programar clases, asignar monitores a diferentes actividades, gestionar horarios y verificar el rendimiento de su gimnasio en términos de membresías y participación en actividades.

4.3. Rol de Monitor

Los monitores son los empleados encargados de llevar a cabo las clases y actividades en los gimnasios. Tienen acceso a las funciones necesarias para crear tanto las tablas de los ejercicios como los ejercicios en sí. Su acceso se centra en las operaciones diarias y la interacción con los socios durante las actividades.

4.4. Rol de Recepcionista

Los recepcionistas son responsables de la atención al cliente y la gestión del flujo de personas en el gimnasio. Tienen acceso para gestionar el registro de nuevos socios, procesar el historial de pagos y mantener actualizada la información de los socios en el sistema. Su enfoque principal es la interacción directa con los socios en la recepción del gimnasio.

4.5. Rol de Socio

Los socios son los usuarios finales de la aplicación. Tienen acceso a funciones como inscribirse en clases y ver su historial de actividades, realizar pagos. Su acceso se limita a las funciones relacionadas con su participación y membresía en el gimnasio.

Cada rol tiene un conjunto específico de permisos y funciones dentro de la aplicación web, lo que permite una distribución adecuada de responsabilidades y acceso de acuerdo con las necesidades y responsabilidades de cada usuario en la cadena de gimnasios.

5. Análisis de requisitos

En este punto se detallarán las fases que han intervenido en el análisis de nuestra aplicación.

Para ello, se definirán los requisitos funcionales y no funcionales, así como el diagrama de casos de uso del sistema.

5.1. Requisitos funcionales

Dentro de este apartado describiremos las diferentes interacciones que tendrán los usuarios con nuestra aplicación.

5.1.1. Inicio de sesión / Cerrar sesión

R1 - 1	El usuario accederá a la pantalla de "Login", que es la primera cuando no tienes la sesión iniciada.
R1 - 2	El usuario deberá introducir el correo electrónico y su contraseña.
R1 - 3	El sistema procederá a validar los datos.
R1 - 4	En caso de que los datos sean incorrectos, el sistema mostrará un mensaje de error.
R1 - 5	Si se validan correctamente los datos, se navegará a la página principal.
R1 - 6	Según el rol del usuario, aparecen unas opciones u otras en el menú.
R1 - 7	El usuario puede cerrar sesión en cualquier momento después de haber "logueado".
R1 - 8	Al pulsarlo se cierra la sesión, volviendo a ser dirigido a la pantalla de "login".

Tabla 1: Requisito funcional Iniciar/Cerrar sesión

5.1.2. Gestión de Gimnasio

R2 - 1	Solo los usuarios con rol de administrador pueden acceder a esta pantalla.
R2 - 2	Se mostrarán todos los gimnasios dados de alta en la aplicación.
R2 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R2 - 4	Se podrá añadir nuevos gimnasios usando la opción de añadir, mostrando de esta forma un formulario para rellenar todos los datos.
R2 - 5	El administrador rellena todos los datos necesarios para dar de alta a un gimnasio.
R2 - 6	El sistema validará los datos que se han introducido.
R2 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de gimnasios. Si no, se muestra un mensaje de error.
R2 - 8	Se pueden modificar los datos de un gimnasio ya dado alta. Para ello, habría que hacer click en el gimnasio que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.

R2 - 9	Los datos del gimnasio deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R2 - 10	Se puede dar de baja a un gimnasio de la aplicación con la opción de eliminar, siempre y cuando ese gimnasio no tenga usuarios asociados.
R2 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 2: Requisito funcional Gestión de Gimnasios

5.1.3. Gestión de Usuarios

R3 - 1	Solo los usuarios con rol de administrador pueden acceder a esta pantalla.
R3 - 2	Se mostrarán todos los usuarios dados de alta en la aplicación, y que están dentro del gimnasio, dependiendo de su rol.
R3 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R3 - 4	Se podrá añadir nuevos usuarios usando la opción de añadir, mostrando de esta forma un formulario para rellenar todos los datos.
R3 - 5	El usuario rellena todos los datos personales del que está dando de alta, así como su rol.
R3 - 6	El sistema validará los datos que se han introducido, como que no se repita el correo electrónico o el teléfono móvil.
R3 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de trabajadores. Si no, se muestra un mensaje de error.
R3 - 8	Se pueden modificar los datos de un usuario ya dado alta. Para ello, habría que hacer click en el usuario que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R3 - 9	Los datos del trabajador deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R3 - 10	Se puede dar de baja a un trabajador de la aplicación con la opción de eliminar, siempre que no tenga datos asociados como por ejemplo ejercicios y evolución.
R3 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 3: Requisito funcional Gestión de Usuarios

5.1.4. Gestión de Jefe

R4 - 1	Solo los usuarios con rol de administrador pueden acceder a esta pantalla.
R4 - 2	Se mostrarán todos los usuarios dados de alta en la aplicación, y que están dentro del gimnasio, dependiendo de su rol.
R4 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R4 - 4	Se podrá añadir nuevos usuarios con el rol de Jefe usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R4 - 5	El usuario rellena todos los datos personales del que está dando de alta, así como su rol.
R4 - 6	El sistema validará los datos que se han introducido, como que no se repita el correo electrónico o el teléfono móvil.
R4 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de jefes. Si no, se muestra un mensaje de error.
R4 - 8	Se pueden modificar los datos de un jefe ya dado alta. Para ello, habría que hacer click en el usuario que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R4 - 9	Los datos del jefe deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitamos.
R4 - 10	Se puede dar de baja a un jefe de la aplicación con la opción de eliminar.
R4 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 4: Requisito funcional Gestión de Jefes

5.1.5. Gestión de Monitor

R5 - 1	Solo los usuarios con rol de administrador y de jefes, pueden acceder a esta pantalla.
R5 - 2	Se mostrarán todos los trabajadores dados de alta en la aplicación, y que están dentro del gimnasio, dependiendo de su rol.
R5 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R5 - 4	Se podrá añadir nuevos usuarios con el rol de Monitor usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R5 - 5	El usuario rellena todos los datos personales del que está dando de alta.
R5 - 6	El sistema validará los datos que se han introducido, como que no se repita el correo electrónico o el teléfono móvil.
R5 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de trabajadores. Si no, se muestra un mensaje de error.
R5 - 8	Se pueden modificar los datos de un monitor ya dado alta. Para ello, habría que hacer click en el usuario que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.

R5 - 9	Los datos del trabajador deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R5 - 10	Se puede dar de baja a un trabajador de la aplicación con la opción de borrar.
R5 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 5: Requisito funcional Gestión de Monitores

5.1.6. Gestión de Recepcionista

R6 - 1	Solo los usuarios con rol de administrador y de jefes, pueden acceder a esta pantalla.
R6 - 2	Se mostrarán todos los trabajadores dados de alta en la aplicación, y que están dentro del gimnasio, dependiendo de su rol.
R6 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R6 - 4	Se podrá añadir nuevos usuarios con el rol de Personal usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R6 - 5	El usuario rellena todos los datos personales del que está dando de alta.
R6 - 6	El sistema validará los datos que se han introducido, como que no se repita el correo electrónico o el teléfono móvil.
R6 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de recepcionistas. Si no, se muestra un mensaje de error.
R6 - 8	Se pueden modificar los datos de un recepcionista ya dado alta. Para ello, habría que hacer click en el usuario que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R6 - 9	Los datos del trabajador deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R6 - 10	Se puede dar de baja a un trabajador de la aplicación con la opción de borrar.
R6 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 6: Requisito funcional Gestión de Recepcionistas

5.1.7. Gestión de Socio

R7 - 1	Solo los usuarios con rol de administrador, recepcionista y de jefes, pueden acceder a esta pantalla.
R7- 2	Se mostrarán todos los socios dados de alta en la aplicación, y que están dentro del gimnasio.
R7 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R7 - 4	Se podrá añadir nuevos usuarios con el rol de Socio usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R7 - 5	El usuario, el cual no sea el propio Socio, rellena todos los datos personales del que está dando de alta.
R7 - 6	El sistema validará los datos que se han introducido, como que no se repita el correo electrónico o el teléfono móvil.
R7 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de socios. Si no, se muestra un mensaje de error.
R7 - 8	Se pueden modificar los datos de un socio ya dado alta. Para ello, habría que hacer click en el usuario que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R7 - 9	Los datos del socio deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R7 - 10	Se puede dar de baja a un trabajador de la aplicación con la opción de borrar.
R7 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 7: Requisito funcional Gestión de Socios

5.1.8. Gestión de Clase

R8 - 1	Solo los usuarios con rol de administrador, jefes y recepcionista pueden acceder a esta pantalla.
R8- 2	Se mostrarán todas las clases dadas de alta en la aplicación, y que están dentro del gimnasio.
R8 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R8 - 4	Se podrá añadir nuevas clases usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R8 - 5	El usuario rellena todos los datos correspondientes para crear una clase.
R8 - 6	El sistema validará los datos que se han introducido.
R8 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de clases. Si no, se muestra un mensaje de error.

R8 - 8	Se pueden modificar los datos de una clase ya creada. Para ello, habría que hacer click en la clase que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R8 - 9	Los datos de la clase deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R8 - 10	Se puede dar de baja a una clase de la aplicación con la opción de borrar.
R8 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 8: Requisito funcional Gestión de Clases

5.1.9. Gestión de Sala

R9 - 1	Solo los usuarios con rol de administrador, jefes y recepcionista pueden acceder a esta pantalla.
R9 - 2	Se mostrarán todas las salas dadas de alta en la aplicación, y que están dentro del gimnasio.
R9 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R9 - 4	Se podrá añadir nuevas salas usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R9 - 5	El usuario rellena todos los datos correspondientes para crear una sala.
R9 - 6	El sistema validará los datos que se han introducido.
R9 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de salas. Si no, se muestra un mensaje de error.
R9 - 8	Se pueden modificar los datos de una sala ya creada. Para ello, habría que hacer click en la sala que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R9 - 9	Los datos de la sala deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R9 - 10	Se puede dar de baja a una sala de la aplicación con la opción de borrar.
R9 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 9: Requisito funcional Gestión de Salas

5.1.10. Calendario de Clases

R10 - 1	Todos los usuarios con rol de administrador, jefes, recepcionista, monitor y socio pueden acceder a esta pantalla.
R10- 2	Se mostrará un calendario con vista semanal o vista diaria.
R10 - 3	Habrà un bloque de texto, el cual se tiene que seleccionar un gimnasio previamente creado.
R10 - 4	Una vez seleccionado el gimnasio, se mostrarán todas las clases creadas en sus respectivos horarios y fechas.
R10 - 5	Al hacer click sobre una clase, se muestran sus datos ya precargados.
R10 - 6	Si se quiere inscribir a una clase, se debe pulsar el botón “Aceptar”, el cual reducirá la capacidad de la clase
R10 - 7	Si se quiere desinscribir de una clase, se debe pulsar el botón “Eliminar”, ya que el botón “Aceptar” estará deshabilitado, el cual aumentará la capacidad de la clase.

Tabla 10: Requisito funcional Calendario de Clase

5.1.11. Gestión de Ejercicios

R11 - 1	Solo los usuarios con rol de administrador, socio y monitor pueden acceder a esta pantalla.
R11- 2	Se mostrarán todos los ejercicios dados de alta en la aplicación, y que están dentro del gimnasio.
R11 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R11 - 4	Se podrá añadir nuevos ejercicios usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R11 - 5	El usuario rellena todos los datos correspondientes para crear un ejercicio.
R11 - 6	El sistema validará los datos que se han introducido.
R11 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de ejercicios. Si no, se muestra un mensaje de error.
R11 - 8	Se pueden modificar los datos de un ejercicio ya creada. Para ello, habría que hacer click en el ejercicio que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R11 - 9	Los datos de la sala deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R11 - 10	Se puede dar de baja a una sala de la aplicación con la opción de borrar.
R11 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

R11 - 12	Los ejercicios creados por el administrador y/o el personal, se tratan como ejercicios por defecto, los cuales ellos si pueden modificar y eliminar, pero un socio no tendría esos permisos
R11 - 13	Los ejercicios creados por un Socio se tratan como que no son ejercicios por defecto, los cuales, si pueden modificar y eliminar, pero el administrador o el personal no tendría esos permisos, porque no le aparecerían esos ejercicios, solo los creado por ellos.

Tabla 11: Requisito funcional Gestión de Ejercicios

5.1.12. Gestión de Categorías de Ejercicios

R13 - 1	Solo los usuarios con rol de administrador, socio y monitor pueden acceder a esta pantalla.
R13- 2	Se mostrarán todas las categorías de ejercicios dados de alta en la aplicación, y que están dentro del gimnasio.
R13 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R13 - 4	Se podrá añadir nuevas categorías de ejercicios usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R13 - 5	El usuario rellena todos los datos correspondientes para crear una categoría de ejercicio.
R13 - 6	El sistema validará los datos que se han introducido.
R13 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de las categorías de ejercicios. Si no, se muestra un mensaje de error.
R13 - 8	Se pueden modificar los datos de una categoría de ejercicio ya creada. Para ello, habría que hacer click en la categoría de ejercicio que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R13 - 9	Los datos de la categoría de ejercicio deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R13 - 10	Se puede dar de baja a una categoría de ejercicio de la aplicación con la opción de borrar.
R13 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 12: Requisito funcional Gestión de Categorías de Ejercicios

5.1.13. Gestión de Horario de Clases

R14 - 1	Solo los usuarios con rol de administrador, jefe y recepcionista pueden acceder a esta pantalla.
R14- 2	Se mostrarán todos los horarios de las clases dadas de alta en la aplicación, y que están dentro del gimnasio.
R14- 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R14 - 4	Se podrá añadir nuevos horarios de clases usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R14 - 5	El usuario rellena todos los datos correspondientes para crear un horario de clases.
R14 - 6	El sistema validará los datos que se han introducido.
R14 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de los horarios de las clases. Si no, se muestra un mensaje de error.
R14 - 8	Se pueden modificar los datos de un horario de clases ya creado. Para ello, habría que hacer click en el horario de clase que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R14 - 9	Los datos del horario de clases deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R14 - 10	Se puede dar de baja a un horario de clases de la aplicación con la opción de borrar.
R14 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 13: Requisito funcional Gestión de Horario de Clase

5.1.14. Gestión de Tablas de Ejercicios

R15 - 1	Solo los usuarios con rol de administrador, socio y monitor pueden acceder a esta pantalla.
R15- 2	Se mostrarán todas las tablas de ejercicios dadas de alta en la aplicación, y que están dentro del gimnasio.
R15 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R15 - 4	Se podrá añadir nuevas tablas de ejercicios usando la opción de añadir, saliendo de esta forma un formulario para rellenar todos los datos.
R15 - 5	El usuario rellena todos los datos correspondientes para crear una tabla de ejercicio.
R15 - 6	El sistema validará los datos que se han introducido.
R15 - 7	Si la validación es satisfactoria, se muestra un mensaje de éxito y se redirige a la pantalla de las tablas de ejercicios. Si no, se muestra un mensaje de error.

R15 - 8	Se pueden modificar los datos de una tabla de ejercicio ya creada. Para ello, habría que hacer click en la tabla de ejercicio que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R15 - 9	Los datos de la tabla de ejercicio deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R15 - 10	Se puede dar de baja a una tabla de ejercicio de la aplicación con la opción de borrar.
R15 - 11	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 14: Requisito funcional Gestión de Tablas de Ejercicios

5.1.15. Gestión de Clases Matriculadas

R16 - 1	Solo los usuarios con rol de administrador y socio pueden acceder a esta pantalla.
R16 - 2	Se mostrarán todas las clases matriculadas donde el usuario se ha inscrito en la aplicación.
R16 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.
R16 - 4	Se puede dar de baja a una clase matriculada de la aplicación con la opción de borrar, siempre y cuando la clase matriculada a borrar sea igual o posterior al día actual.
R16 - 5	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 15: Requisito funcional Gestión de Clases Matriculadas

5.1.16. Gestión de Historial de pago

R17 - 1	Solo los usuarios con rol de administradores, jefe y recepcionista pueden acceder a esta pantalla.
R17- 2	Se mostrarán todos los socios que hayan pagado la cuota del gimnasio en la aplicación.
R17 - 3	Habrà un bloque de texto que actúa como una búsqueda rápida dentro de la tabla, para poder encontrar algo rápidamente.

Tabla 16: Requisito funcional Gestión de Historial de pago

5.1.17. Gestión de Evolución de Ejercicios

R18 - 1	Solo los usuarios con rol de administrador, socio y monitor pueden acceder a esta pantalla.
R18- 2	Se mostrará un pop-up con el listado de tablas de ejercicios tanto las creadas por el socio como las existentes del gimnasio
R18 - 3	Se mostrarán 2 tablas, la primera de ellas, contendrá los ejercicios con sus series, repeticiones y/o distancia objetivo, referente a la tabla previamente seleccionada. La segunda de ella contendrá las series, repeticiones y/o distancias reales, junto al peso utilizado.
R18 - 4	Se podrá añadir nuevas series de un ejercicio, haciendo click en el ejercicio de la primera tabla.
R18 - 8	Se pueden modificar los datos de una serie y ejercicio al ser seleccionados. Para ello, habría que hacer click en la serie de un ejercicio que se quiera editar sus datos, donde aparecerá un modal con los datos a editar.
R18 - 9	Los datos de la serie del ejercicio deben aparecer precargados en la pantalla de edición, pudiendo cambiar los campos que necesitemos.
R18 - 10	Una vez pulsado el botón y cargada la acción, se mostrará un mensaje de éxito.

Tabla 17: Requisito funcional Gestión de Evolución de Ejercicios

5.1.18. Gestión de Pasarela de pago

R19 - 1	Solo los usuarios con rol de socio pueden acceder a esta pantalla.
R19- 2	Se mostrará automáticamente tras el login del socio, siempre y cuando no haya realizado el pago del mes actual.
R19 - 3	El socio tendrá 15 días hábiles del mes actual para realizar el pago y, si este no es realizado perderá el acceso a la aplicación.

Tabla 18: Requisito funcional Gestión de Pasarela de pago

5.2. Requisitos no funcionales

Los requisitos no funcionales especifican las características generales y los criterios que interfieren en las operaciones del sistema.

5.2.1. Seguridad

RN1 - 1	El usuario debe de autenticarse para poder iniciar sesión.
RN1 - 2	Todos los campos de los formularios serán validados por el sistema.
RN1 - 3	Las opciones del menú que requieran tener el rol de algún tipo de administrador quedarán invisibles e inalcanzables para los usuarios que no lo sean.
RN1 - 4	En la base de datos, para cada usuario, se guardará un 'passwordSalt' generado automáticamente en el momento de darle de alta, y un 'passwordHash', que es el resultado de aplicar este "salt" a la contraseña introducida.
RN1 - 5	En la página web, una vez "logueado", no hará falta volver a iniciar sesión hasta que no se cierre.

Tabla 19: Requisito no funcional Seguridad

5.2.2. Mantenibilidad y sistema

RN2 - 1	Todos los datos serán almacenados en una base de datos MySQL.
RN2 - 2	Será necesaria la conexión a internet para usar la aplicación.
RN2 - 3	La página web estará disponible para los navegadores Internet Explorer, Microsoft Edge, Google Chrome y Firefox.

Tabla 20: Requisito no funcional Mantenibilidad y Sistema

5.2.3. Interfaz y rendimiento

RN3 - 1	La interfaz deberá ser minimalista y clara.
RN3 - 2	Deberá ser fácil de usar y con una curva de aprendizaje pequeña.
RN3 - 3	La aplicación deberá tener un alto índice de disponibilidad para asegurar que los trabajadores y socios puedan utilizarla.
RN3 - 4	Los tiempos de respuesta de la aplicación deberán ser lo suficientemente cortos para un buen uso de la aplicación.
RN3 - 5	Podrán existir varios usuarios distintos conectados a la aplicación en el mismo momento.

Tabla 21: Requisito no funcional interfaz y rendimiento

5.3. Diagramas de casos de usos

En este apartado vamos a observar los diferentes casos de uso de nuestra página web. Contaremos con cinco actores principales, Administrador, Jefe, Recepción, Monitor y Socio.

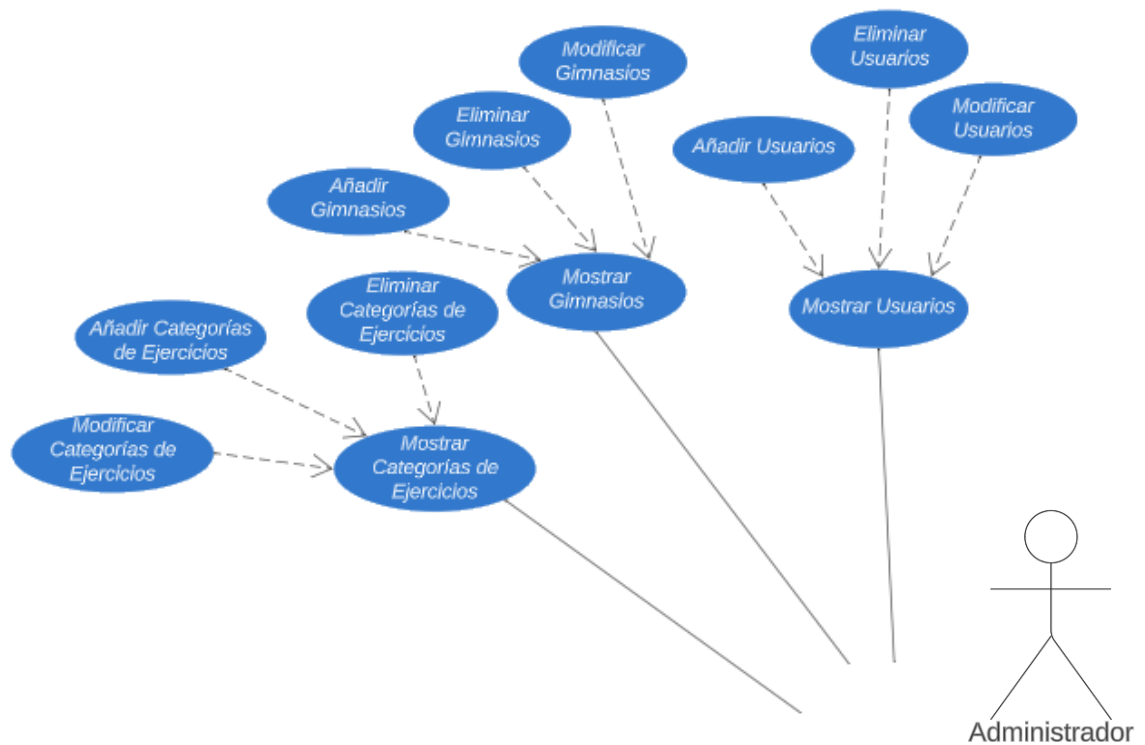


Ilustración 4: Diagrama de casos de uso del rol de Administrador (parte 1)

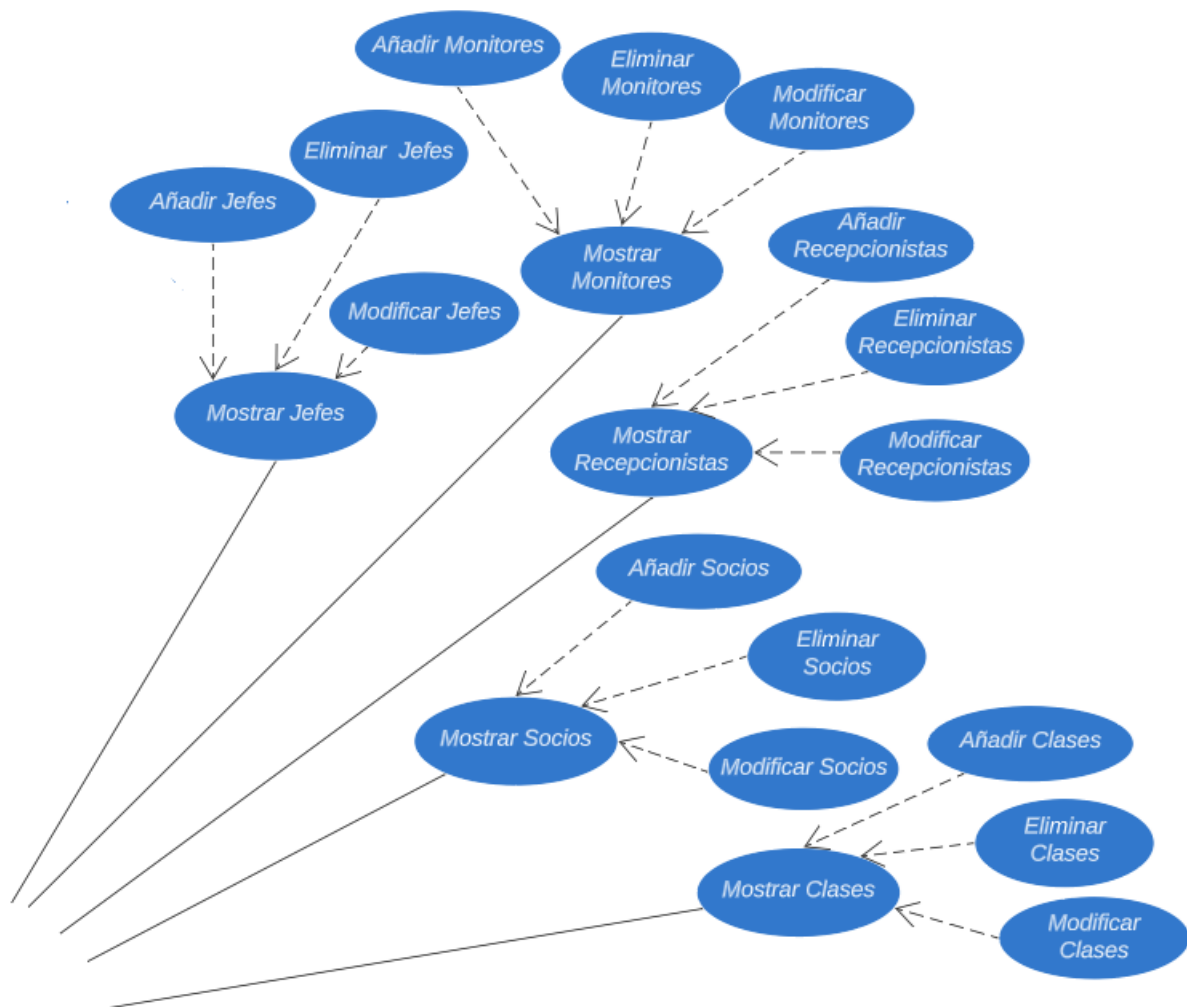


Ilustración 5: Diagrama de casos de uso del rol de Adminitrador (parte 2)

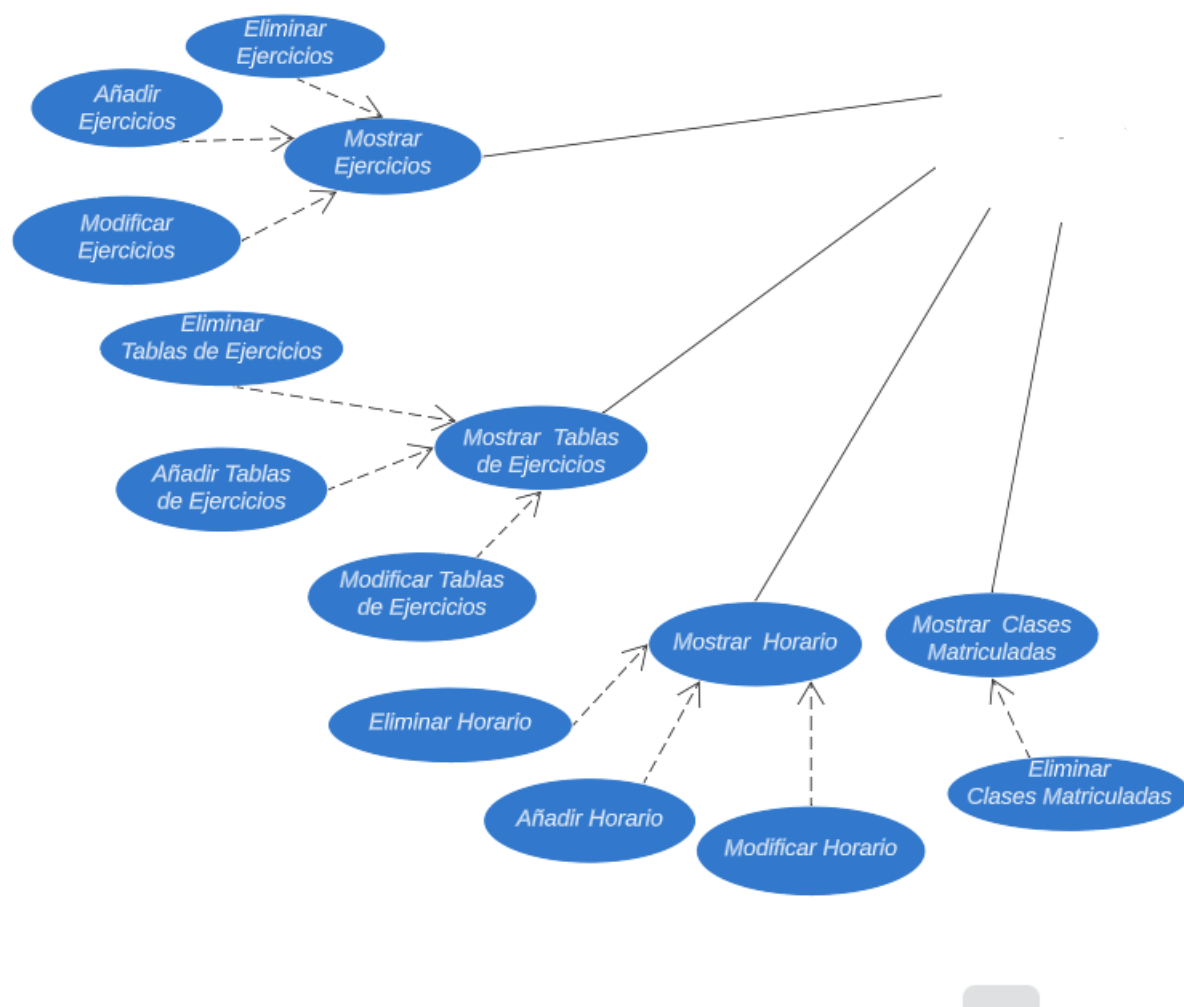


Ilustración 6: Diagrama de casos de uso del rol de Adminitrador (parte 3)

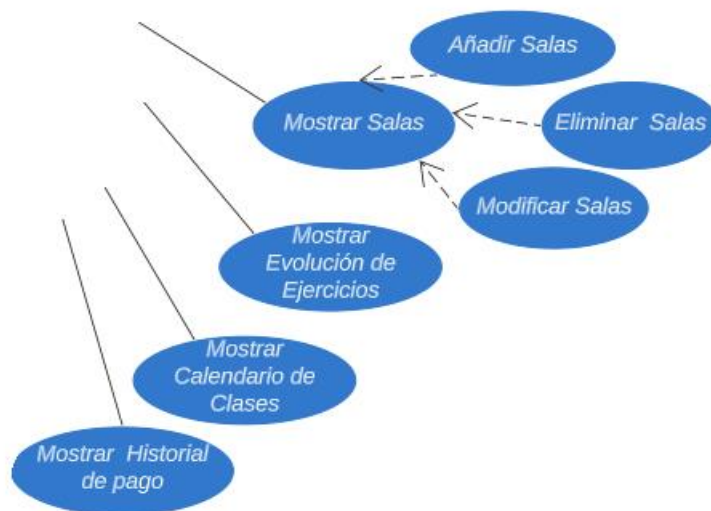


Ilustración 7: Diagrama de casos de uso del rol de Adminitrador (parte 4)



Ilustración 8: Diagrama de casos de uso del rol de Jefe



Ilustración 9: Diagrama de casos de uso del rol de Recepcionista

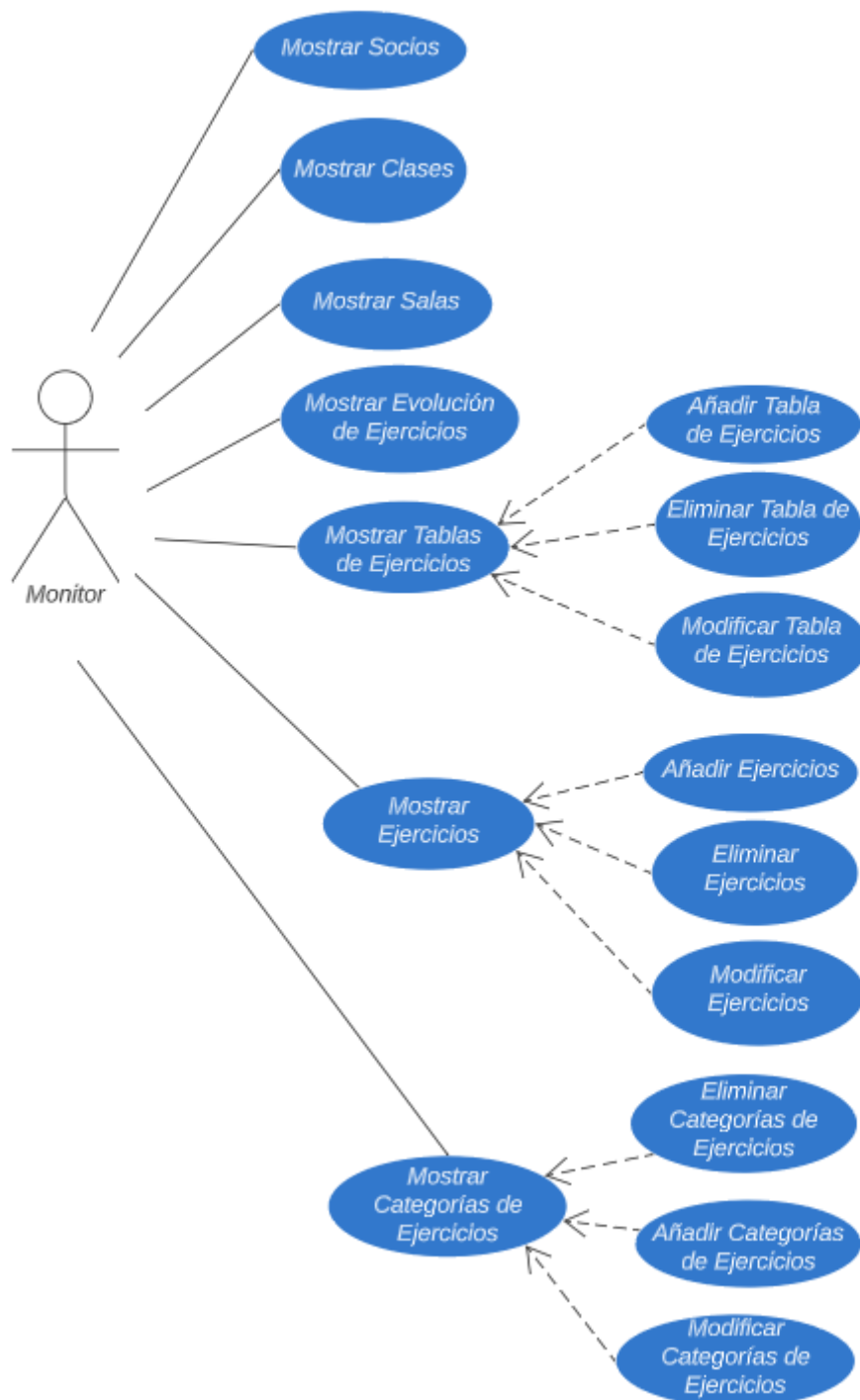


Ilustración 10: Diagrama de casos de uso del rol de Monitor

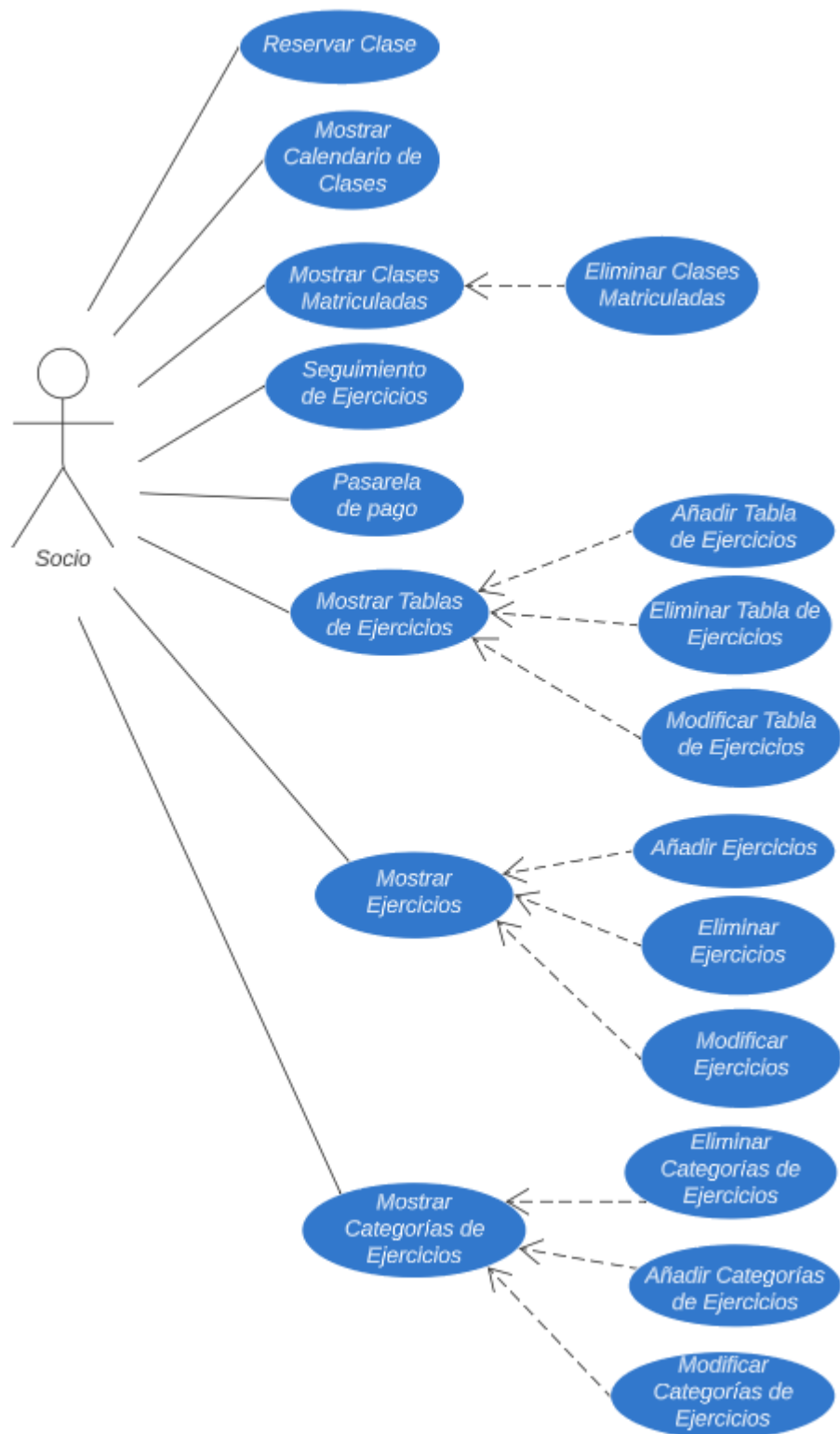
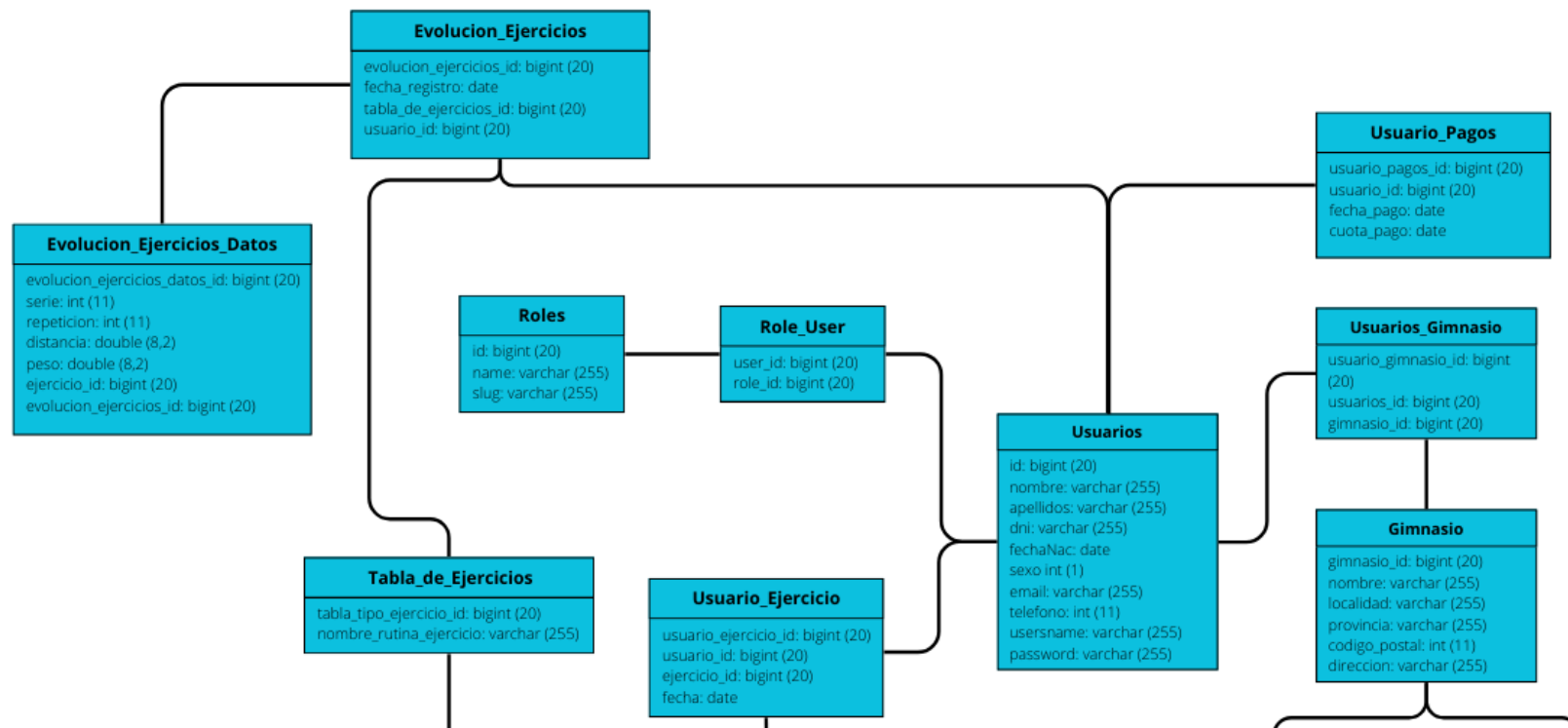


Ilustración 11: Diagrama de casos de uso del rol de Socio

6. Diseño

6.1. Diagrama Entidad/Relación

En la imagen podemos observar las distintas entidades que componen la base de datos, así como las relaciones entre las mismas.



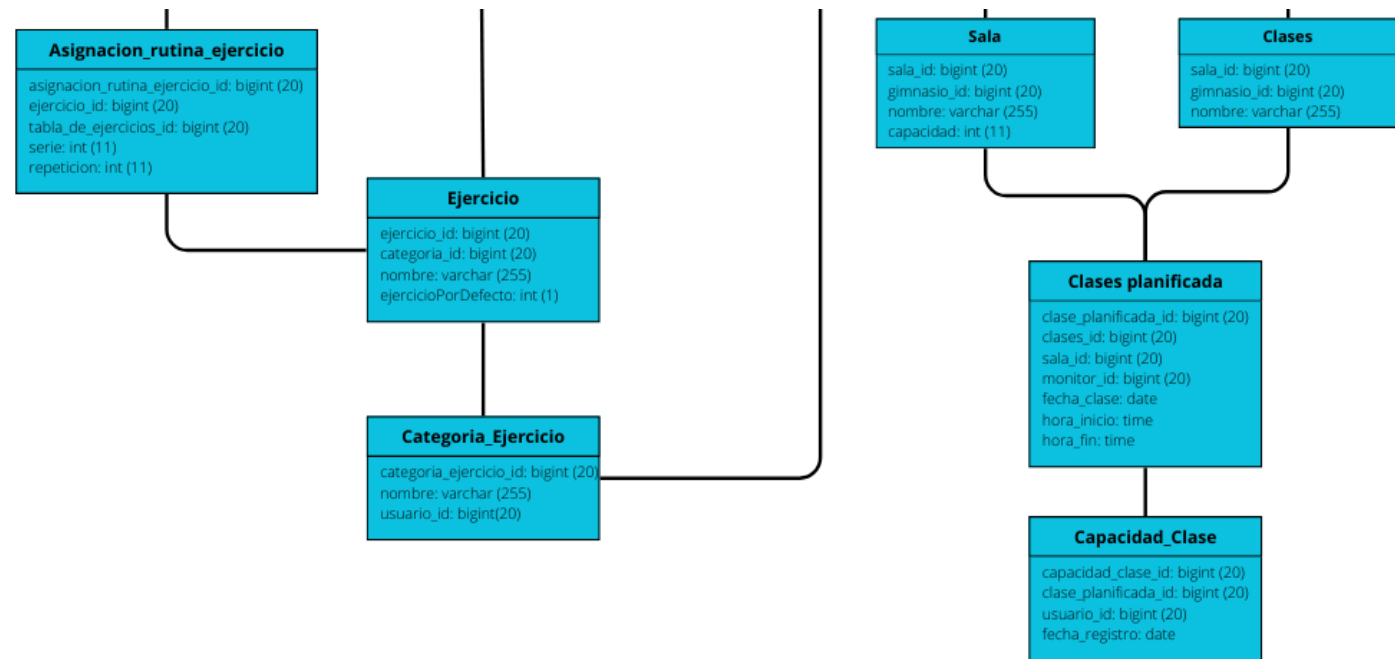


Ilustración 12: Diagrama Entidad/Relación de la BD

6.2. Descripción del modelo

A continuación, vamos a describir el modelo de nuestra base de datos. En la tabla de Usuario guardaremos todos los usuarios, tengan el rol que tengan, y este se podrá saber por un atributo específico. También se puede observar claramente como los gimnasios van a ser creados por el administrador. Los trabajadores, los cuales, han debido de ser dados de alta por el jefe. Por último, los trabajadores (tanto personal como recepcionista) van a ser los encargados de realizar los diferentes ejercicios, tabla de ejercicios y las clases.

Veamos con detalle los datos presentes en cada tabla:

6.2.1. Tabla 'Usuario'

Se almacenan todos los datos de los usuarios.

- **id:** Id del usuario.
- **nombre:** Nombre del usuario.
- **apellidos:** Apellidos del usuario.
- **dni:** DNI del usuario
- **fechaNac:** Fecha de nacimiento del usuario.
- **sexo:** Sexo del usuario.
- **email:** Dirección de correo electrónico del usuario.
- **teléfono:** Número de teléfono móvil del usuario.
- **username:** Username del usuario.
- **password:** Hash creado automáticamente para cifrar la contraseña.

6.2.2. Tabla 'Roles'

Se almacenan todos los datos de los roles.

- **id:** Id del rol.
- **name:** Nombre del rol.
- **slug:** Nickname del rol.

6.2.3. Tabla 'Role_User'

Se almacenan todos los datos de la relación entre los roles y los usuarios.

- **user_id**: Id del usuario.
- **role_id**: Id del rol.

6.2.4. Tabla 'Gimnasio'

Se almacenan todos los datos de los gimnasios.

- **gimnasio_id**: Id del gimnasio.
- **nombre**: Nombre del gimnasio.
- **direccion**: Dirección del gimnasio.
- **localidad**: Localidad del gimnasio.
- **provincia**: Provincia del gimnasio.

6.2.5. Tabla 'Usuario_Gimnasio'

Se almacenan todos los datos de la relación entre los usuarios y los gimnasios.

- **usuario_id**: Id del usuario.
- **gimnasio_id**: Id del gimnasio.
- **usuario_gimnasio_id**: Id de la relación entre el gimnasio y el usuario.

6.2.6. Tabla 'Sala'

Se almacenan todos los datos de las salas de los gimnasios

- **sala_id**: Id de la sala.
- **nombre**: Nombre de la sala.
- **capacidad**: Capacidad de la sala.
- **gimnasio_id**: Id del gimnasio

6.2.7. Tabla 'Clases'

Se almacenan todos los datos de las clases de los gimnasios

- **clases_id:** Id de la clase.
- **nombre:** Nombre de la sala.
- **gimnasio_id:** Id del gimnasio.

6.2.8. Tabla 'Clase_Planificada'

Se almacenan todos los datos de la relación entre las clases y las salas.

- **clase_planificada_id:** Id de la clase planificada.
- **fecha_clase:** Fecha de la clase planificada.
- **hora_inicio:** Hora inicio de la clase planificada
- **hora_fin:** Hora fin de la clase planificada
- **clases_id:** Id de la clase
- **sala_id:** Id de la sala.
- **monitor_id:** Id del monitor (personal) que imparte la clase planificada

6.2.9. Tabla 'Capacidad_Clase'

Se almacenan todos los datos de la capacidad de las clases.

- **capacidad_clase_id:** Id de la capacidad.
- **fecha_registro:** Fecha de registro del usuario a esa clase planificada.
- **clase_planificada_id:** Id de la clase planificada.
- **usuario_id:** Id del usuario.

6.2.10. Tabla 'Ejercicio'

Se almacenan todos los datos de los ejercicios

- **ejercicio_id:** Id del ejercicio
- **nombre:** Nombre del ejercicio.
- **ejercicioPorDefecto:** Saber si un ejercicio es por defecto (del mismo gimnasio) o añadido por un socio
- **categoria_id:** Id de la categoría del ejercicio.

6.2.11. Tabla 'Usuario_Ejercicio'

Se almacenan todos los datos de la relación entre los ejercicios y los usuarios.

- **usuario_ejercicio_id:** Id del ejercicio
- **fecha:** Fecha de creación de los ejercicios introducidos por el usuario.
- **usuarios_id:** Id del usuario
- **ejercicio_id:** Id del ejercicio

6.2.12. Tabla 'Categoria_Ejercicio'

Se almacenan todos los datos de las categorías de los ejercicios

- **categoria_ejercicio_id:** Id de la categoría de ejercicio
- **nombre:** Nombre del tipo de ejercicio.
- **Usuario_id:** Id del usuario que crea la categoría

6.2.13. Tabla 'Tabla_de_Ejercicios'

Se almacenan todos los datos de las tablas de los ejercicios

- **tabla_de_ejercicio_id:** Id de la tabla/rutina de ejercicios
- **nombre_rutina_ejercicio:** Nombre de la tabla/rutina de ejercicios.

6.2.14. Tabla 'Asignación_Rutina_Ejercicio'

Se almacenan todos los datos de la relación entre las tablas de los ejercicios y los ejercicios.

- **asignacion_rutina_ejercicio_id:** Id de la asignación de la rutina de ejercicios.
- **ejercicio_id:** Id del ejercicio.
- **tabla_de_ejercicios_id:** Id de la tabla de ejercicios.
- **serie:** Número de series objetivo del ejercicio.
- **repeticion:** Número de repeticiones objetivo del ejercicio

6.2.15. Tabla 'Usuario_Pagos'

Se almacenan todos los datos de los pagos realizados por los usuarios.

- **usuario_pagos_id:** Id de los pagos de los usuarios.
- **usuario_id:** Id del usuario.
- **fecha_pago:** Fecha del pago realizado.
- **cuota_pago:** Cuota del pago.

6.2.16. Tabla 'Evolucion_Ejercicios'

Se almacenan todas las evoluciones de los ejercicios de los usuarios.

- **evolucion_ejercicios_id:** Id de los pagos de los usuarios.
- **tabla_de_ejercicios_id:** Id de la tabla de ejercicios.
- **usuario_id:** Id del usuario.
- **fecha_registro:** Fecha registro del día de evolución.

6.2.17. Tabla 'Evolucion_Ejercicios_Datos'

Se almacenan todos los datos reales de las evoluciones de los ejercicios de los usuarios.

- **evolucion_ejercicios_datos_id:** Id de los pagos de los usuarios.
- **ejercicio_id:** Id de los ejercicios.
- **evolucion_ejercicios_id:** Id del usuario.
- **serie:** Series reales del ejercicio.
- **repetition:** Repeticiones reales del ejercicio.
- **peso:** Pesos reales del ejercicio.

7. Arquitectura de la aplicación

Nuestra aplicación web sigue una arquitectura cliente-servidor. Se va a necesitar un servidor de base de datos y un servidor web. En nuestro caso, para el de base de datos hemos escogido MySQL y para el hosting, hemos optado por Hostinger cuyo servidor web es LiteSpeed.

El flujo será el siguiente:

1. El cliente hará una petición a nuestro servidor web, en el cual está alojado nuestro back-end. Los controladores de este serán los encargados de recoger las peticiones.
2. Posteriormente, a través de los repositorios que tenemos en el back-end, se harán las peticiones necesarias al servidor de base de datos para recuperar o persistir la información que necesite la BBDD.
3. El servidor de base de datos nos devolverá los resultados de la petición a nuestro servidor web.
4. Por último, el servidor web completará las diferentes vistas en HTML con los datos que hemos recogido, estando estas ya a disposición del cliente.

7.1. Aplicación web

Dentro de la aplicación web, debemos diferenciar entre el front-end y el back-end. En el front-end se ha utilizado HTML y CSS y en el back-end se ha utilizado PHP con su framework Laravel.

A continuación, vamos a explicar la razón por la que hemos optado por estas tecnologías. En primer lugar, vamos a hablar del por qué hemos escogido PHP y Laravel.

Las principales ventajas que tiene PHP son las siguientes:

- **Amplia disponibilidad:** PHP es uno de los lenguajes de programación más populares y ampliamente utilizados en el desarrollo web. Esto significa que hay una gran cantidad de recursos, documentación y soporte disponibles en línea, así como una comunidad activa de desarrolladores.

- **Facilidad de aprendizaje:** PHP tiene una curva de aprendizaje relativamente baja, especialmente para aquellos que ya tienen experiencia en lenguajes de programación similares como C o Java. Su sintaxis es fácil de entender y la comunidad de PHP ofrece una gran cantidad de tutoriales y ejemplos para ayudar a los nuevos desarrolladores a ponerse en marcha rápidamente.
- **Integración con bases de datos:** PHP cuenta con una amplia compatibilidad con una variedad de sistemas de gestión de bases de datos, lo que facilita la interacción con ellos. Esto permite a los desarrolladores crear aplicaciones web dinámicas y acceder a los datos almacenados en la base de datos de manera eficiente.
- **Flexibilidad:** PHP es un lenguaje muy flexible que se adapta a diferentes necesidades. Se puede usar tanto para crear sitios web pequeños y simples como para desarrollar aplicaciones web complejas y de gran escala. Además, es compatible con una amplia gama de servidores web y sistemas operativos.

Las principales ventajas que tiene Laravel son las siguientes:

- **Eficiencia en el desarrollo:** Laravel simplifica muchas tareas comunes en el desarrollo web, como enrutamiento, gestión de bases de datos, manejo de sesiones y autenticación. Proporciona una sintaxis elegante y una API intuitiva que permite a los desarrolladores escribir código de manera más rápida y eficiente.
- **Arquitectura MVC:** Laravel sigue el patrón de diseño Modelo-Vista-Controlador (MVC), lo que ayuda a mantener el código organizado y separado en capas. Esto facilita la escalabilidad y el mantenimiento del proyecto a medida que crece.
- **Comunidad y ecosistema:** Laravel cuenta con una comunidad activa y comprometida de desarrolladores que contribuyen con una amplia variedad de paquetes y extensiones para el framework. Esto facilita la incorporación de características adicionales a tus aplicaciones y ahorra tiempo en el desarrollo.
- **Seguridad:** Laravel incluye características de seguridad integradas, como protección contra ataques de inyección SQL, cross-site scripting (XSS) y falsificación de solicitudes entre sitios (CSRF). Además, proporciona métodos para encriptar contraseñas y proteger los datos sensibles.

A continuación, explicaremos las ventajas sobre las tecnologías utilizadas para el front-end.

Las principales ventajas que tiene HTML son las siguientes:

- **Estructura y semántica:** HTML proporciona una estructura básica para construir páginas web. Permite organizar el contenido de manera semántica, lo que facilita la comprensión del contenido tanto para los desarrolladores como para los motores de búsqueda.
- **Compatibilidad y accesibilidad:** HTML es un estándar ampliamente aceptado y compatible con todos los navegadores web modernos. Además, permite mejorar la accesibilidad de las páginas web al proporcionar etiquetas y atributos específicos para describir elementos y mejorar la experiencia de los usuarios con discapacidades.
- **Integración con otros lenguajes:** HTML se puede combinar fácilmente con otros lenguajes de programación y tecnologías, como CSS para estilos y JavaScript para funcionalidad interactiva. Esto permite crear sitios web más dinámicos y atractivos.
- **Fácil de aprender:** HTML tiene una sintaxis simple y fácil de aprender, lo que permite a los principiantes comenzar a construir páginas web rápidamente. Además, hay una gran cantidad de recursos y tutoriales en línea disponibles para ayudar en el proceso de aprendizaje.

Las principales ventajas que tiene CSS son las siguientes:

- **Diseño visual:** CSS se utiliza para controlar la presentación y el aspecto visual de las páginas web. Permite definir estilos, colores, fuentes, diseños y efectos visuales, lo que permite crear sitios web atractivos y personalizados.
- **Separación de la presentación y el contenido:** CSS permite separar la estructura y el contenido de una página web de su diseño visual. Esto facilita la actualización y el mantenimiento de los estilos, ya que los cambios en la hoja de estilos CSS se aplican automáticamente a todas las páginas que la utilizan.
- **Eficiencia y rendimiento:** CSS utiliza reglas y selectores para aplicar estilos a elementos HTML, lo que permite un enfoque más eficiente y escalable en comparación con aplicar estilos individualmente a cada elemento. Además,

los estilos CSS se pueden almacenar en archivos externos y se pueden almacenar en caché, lo que mejora el rendimiento de las páginas web.

- **Responsividad y adaptabilidad:** CSS proporciona características y técnicas para crear diseños responsivos, que se adaptan automáticamente a diferentes dispositivos y tamaños de pantalla. Esto permite que los sitios web sean accesibles y se vean bien en una amplia gama de dispositivos, desde computadoras de escritorio hasta dispositivos móviles.
- **Animaciones y transiciones:** CSS ofrece capacidades para crear animaciones y transiciones fluidas en elementos HTML, lo que permite agregar interactividad y mejorar la experiencia del usuario en los sitios web.

7.2. Base de datos

El motor de base de datos empleado es MySQL 5.7. Dentro de la base de datos se encuentran el operacional de la aplicación, que son las tablas de respaldo de las entidades.

Para contextualizar, un ORM (Object Relational Mapping) es una técnica de programación que nos ayuda a manipular y consultar la información almacenada dentro de una base de datos usando programación orientada a objetos. Se encargan de la conexión y también de manejar todo con base en modelos o entidades.

A continuación, como hemos hecho con las otras partes de la arquitectura, vamos a exponer las principales ventajas que nos han hecho escoger este motor de base de datos por encima de otros importantes.

- **Es rápida.** La cualidad más destacada por quienes desarrollan MySQL es su velocidad y así es como el software fue diseñado desde un principio, pensando en la rapidez.
- **No es caro.** MySQL es gratis bajo la licencia GPL de código abierto.
- **Fácil de usar.** Puedes construir e interactuar con una base de datos MySQL, siguiendo simples reglas en el lenguaje SQL.
- **Mucha disponibilidad.** Está disponible en casi todos los proveedores de hosting.
- **Es seguro.** El sistema flexible de autorización de MySQL permite algunos o todos los privilegios de base de datos a usuarios específicos o grupos de ellos.

7.3. Interfaces de usuario

Nuestra página web va a ser usada por todo tipo de personas con diferentes responsabilidades dentro de los gimnasios, desde gente joven a gente adulta. Se ha buscado un diseño minimalista y sencillo.

Debido a que en todo momento hemos querido ofrecer una aplicación multiplataforma, decidimos que la interfaz fuera totalmente responsiva. Esto, en la página web, se ha llevado a cabo con la librería Bootstrap.

7.3.1. Responsive Web Design

Se refiere a una técnica de diseño y desarrollo de sitios web que busca proporcionar una experiencia óptima al usuario, independientemente del dispositivo que esté utilizando para acceder al sitio. La idea principal detrás del diseño web responsivo es crear un diseño flexible y adaptable que se ajuste automáticamente al tamaño de la pantalla y a las capacidades del dispositivo, como computadoras de escritorio, laptops, tablets y teléfonos móviles.

En lugar de crear diferentes versiones de un sitio web para diferentes dispositivos, el diseño web responsivo utiliza un conjunto de técnicas y principios como media queries, rejillas fluidas e imágenes flexibles para permitir que el diseño se adapte y reorganice automáticamente en función del tamaño de la pantalla. Esto significa que el contenido del sitio web se ajustará y se presentará de una manera óptima sin importar si el usuario está viendo el sitio en una pantalla grande o en un dispositivo móvil con una pantalla más pequeña.

El objetivo final del diseño web responsivo es proporcionar una experiencia de usuario consistente y agradable, independientemente del dispositivo que se esté utilizando. Al adaptarse al dispositivo del usuario, el diseño web responsivo mejora la usabilidad y la legibilidad del contenido, evitando la necesidad de desplazamiento horizontal o zoom excesivo. Además, también ayuda en términos de optimización para motores de búsqueda (SEO) y facilita la administración y el mantenimiento del sitio web al tener una sola versión de este.

En resumen, el Responsive Web Design se trata de diseñar y desarrollar sitios web que sean flexibles y se adapten automáticamente a diferentes tamaños de pantalla y dispositivos, brindando una experiencia óptima al usuario.



Ilustración 13: Ejemplo Diseño web Responsive

7.4. Lenguajes de programación y herramientas

7.4.1. MySQL

Es un sistema de gestión de bases de datos relacionales (RDBMS, por sus siglas en inglés) muy popular y ampliamente utilizado. RDBMS se refiere a un software diseñado para administrar, almacenar y recuperar datos organizados en tablas relacionadas entre sí.

MySQL se destaca por ser un sistema de base de datos de código abierto, lo que significa que su código fuente está disponible para su estudio, modificación y distribución. Es ampliamente utilizado en aplicaciones web y otros entornos donde se requiere almacenar y administrar grandes cantidades de datos.

7.4.2. HTML

Es el lenguaje de marcado estándar utilizado para crear y estructurar contenido en la web. Es la base fundamental de cualquier página web y se utiliza para definir la estructura y el formato del contenido, como texto, imágenes, enlaces y otros elementos.

En términos muy simples, HTML es un conjunto de etiquetas o elementos que rodean el contenido y le dan significado y formato. Estas etiquetas se escriben en un archivo HTML y son interpretadas por el navegador web para mostrar el contenido correctamente.

7.4.3. CSS

Es un lenguaje de estilo utilizado para definir la apariencia visual y el diseño de una página web. Se utiliza en combinación con HTML para separar el contenido de la presentación, lo que permite un mayor control sobre cómo se ve y se muestra el contenido en el navegador.

En términos muy simples, CSS se encarga de definir los colores, fuentes, tamaños, espacios, márgenes y otros estilos visuales de los elementos HTML en una página web. Con CSS, se pueden establecer reglas y propiedades para controlar el diseño y la presentación de una página, aplicando estilos a elementos individuales o a grupos de elementos.

7.4.4. JavaScript

Es un lenguaje de programación de alto nivel que se utiliza principalmente en el desarrollo web para crear interactividad y funcionalidad en las páginas web. Es un lenguaje de scripting que se ejecuta en el navegador del usuario y permite realizar acciones y manipular el contenido de una página web en tiempo real.

En resumen, JavaScript se utiliza para agregar comportamiento a las páginas web. Con JavaScript, se pueden realizar tareas como validar formularios, responder a eventos del usuario (como hacer clic en un botón), crear animaciones, realizar llamadas a servidores para obtener datos actualizados (mediante AJAX) y mucho más.

7.4.5. Laravel

Es un framework de desarrollo web de código abierto y basado en PHP. Proporciona un conjunto de herramientas y bibliotecas que permiten desarrollar aplicaciones web de manera rápida y eficiente.

En resumen, es un marco de trabajo que simplifica y acelera el proceso de construcción de aplicaciones web. Ofrece una estructura organizada y coherente,

junto con numerosas funciones y características integradas, lo que facilita el desarrollo de aplicaciones complejas.

7.4.6. Bootstrap

Es un framework de desarrollo front-end de código abierto que proporciona un conjunto de herramientas y estilos predefinidos para facilitar la creación de sitios web y aplicaciones web responsivas y visualmente atractivas.

En resumen, es una biblioteca de HTML, CSS y JavaScript que permite diseñar y estructurar rápidamente el aspecto visual y la disposición de los elementos en una página web. Proporciona una colección de componentes reutilizables, como botones, formularios, navegación y cuadros modales, que se pueden utilizar para crear interfaces de usuario interactivas y atractivas.

7.4.7. AdminLTE

Es un popular framework de interfaz de usuario (UI) de código abierto basado en Bootstrap. Está diseñado específicamente para crear paneles de administración y aplicaciones de back-end con una interfaz de usuario profesional y fácil de usar.

De forma breve, proporciona un conjunto de plantillas y componentes predefinidos que permiten desarrollar rápidamente interfaces de administración atractivas y funcionales. Está construido sobre Bootstrap y aprovecha sus características de diseño responsivo y compatibilidad con múltiples navegadores.

7.4.8. Visual Studio Code

Es un editor de código fuente desarrollado por Microsoft. Es un entorno de desarrollo ligero y altamente personalizable que admite múltiples lenguajes de programación y proporciona una amplia gama de características y extensiones para facilitar la escritura y el manejo de código.

De forma resumida, es una herramienta que permite a los desarrolladores escribir y editar código de manera eficiente. Ofrece una interfaz de usuario intuitiva y fácil de usar, con características como resaltado de sintaxis, autocompletado inteligente, depuración integrada, control de versiones y mucho más.

7.4.9. Stripe

Es una plataforma de pagos en línea que permite a las empresas aceptar pagos con tarjeta de crédito y débito de manera segura y sencilla a través de internet. Proporciona una infraestructura de pagos completa que incluye herramientas de procesamiento de transacciones, seguridad, prevención de fraudes y gestión de suscripciones.

En resumen, es un servicio que facilita a las empresas la integración de pagos en línea en sus sitios web y aplicaciones. Proporciona una interfaz de programación de aplicaciones (API) que permite a los desarrolladores conectarse a la plataforma y realizar transacciones de pago sin tener que lidiar directamente con la complejidad de los sistemas de pagos tradicionales.

7.4.10 Hostinger

Es una empresa de hosting y servicios de dominio que proporciona servicios de alojamiento web accesibles y confiables. Ofrece planes de alojamiento compartido, alojamiento en la nube, alojamiento WordPress, servidores virtuales privados (VPS) y más.

En términos muy simples, permite a las personas y empresas alojar sus sitios web en servidores en línea para que sean accesibles en Internet. Proporciona servicios de alojamiento que incluyen almacenamiento de archivos, bases de datos, correo electrónico y otros recursos necesarios para mantener un sitio web en funcionamiento.

7.4.11 LiteSpeed

Es un software de servidor web de alto rendimiento que puede utilizarse para servir sitios web y aplicaciones en línea. Proporciona ventajas en términos de velocidad, eficiencia y escalabilidad. Algunos proveedores de hosting utilizan LiteSpeed como alternativa al servidor web Apache, ya que puede mejorar significativamente el rendimiento de los sitios web y las aplicaciones al reducir la carga del servidor y ofrecer tiempos de carga más rápidos.

8. Diseño de Interfaces de usuario

Dentro de este apartado se va a exponer los diseños de las diferentes interfaces de usuario de las distintas pantallas que tiene la aplicación.

8.1. Pantalla de Inicio

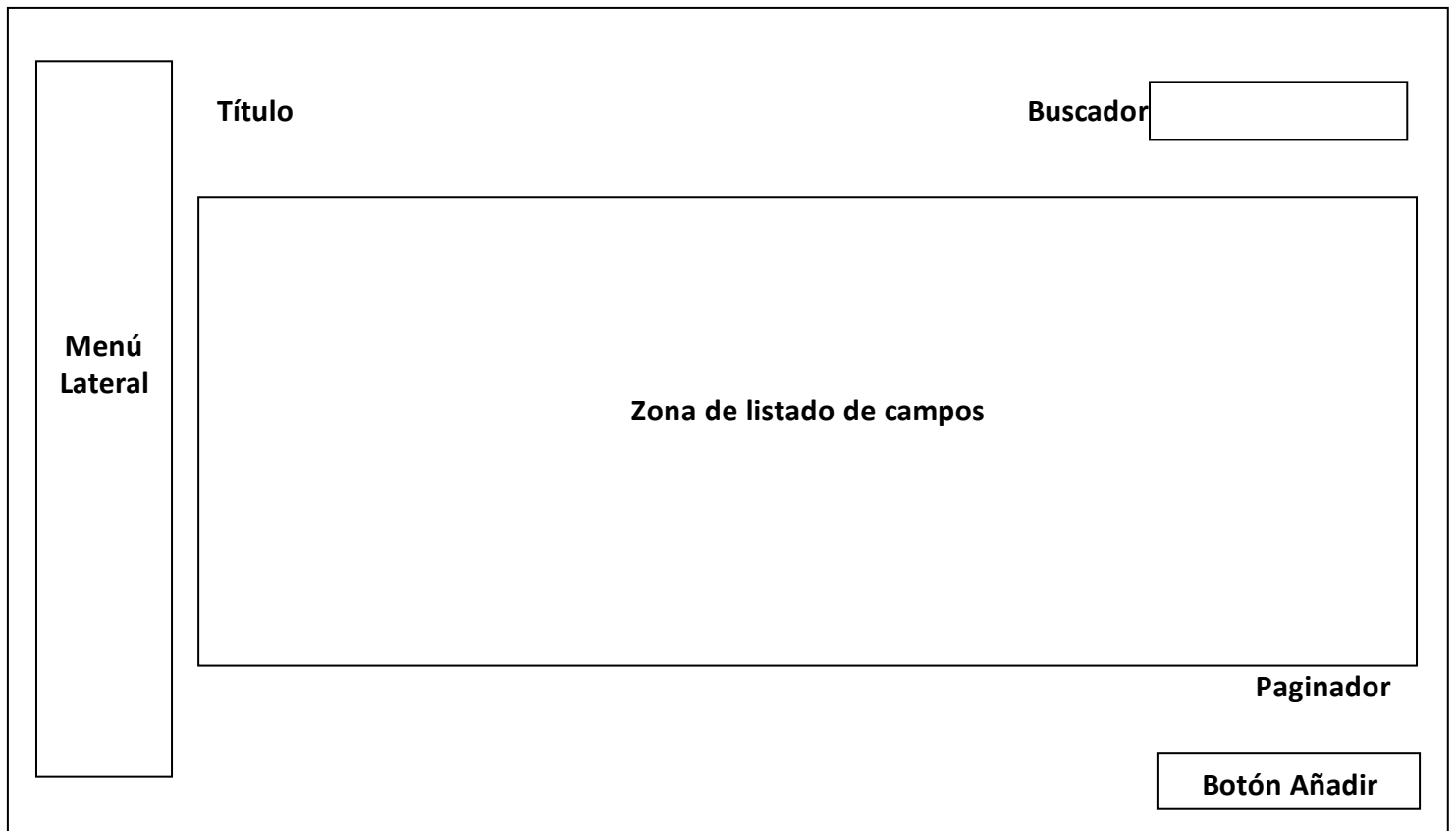


Ilustración 14: Diseño de Pantalla de Inicio

8.2. Pantalla de Inserción con selector

Título

Zona de listado de campos

Selector

Botón Cancelar **Botón Aceptar**

Ilustración 15: Diseño de Pantalla de Inserción con selector

8.3. Pantalla de Inserción sin selector

Título

Zona de listado de campos

Botón Cancelar **Botón Aceptar**

Ilustración 16: Diseño de Pantalla de Inserción sin selector

8.4. Pantalla de Eliminación

Título

Texto

Botón Cancelar **Botón Aceptar**

Ilustración 17: Diseño de Pantalla de Eliminación

8.5. Pantalla de Actualización con selector

Título

Zona de listado de campos

Selector

Botón Eliminar **Botón Cancelar** **Botón Aceptar**

Ilustración 18: Diseño de Pantalla de Actualización con selector

8.6. Pantalla de Actualización sin selector

Título

Zona de listado de campos

Botón Eliminar

Botón Cancelar

Botón Aceptar

Ilustración 19: Diseño de Pantalla de Actualización sin selector

8.7. Pantalla de Pasarela de pago

Título

Campo

Botón Pagar

Ilustración 20: Diseño de Pantalla de Pasarela de pago

8.8. Pantalla de Calendario de Clases

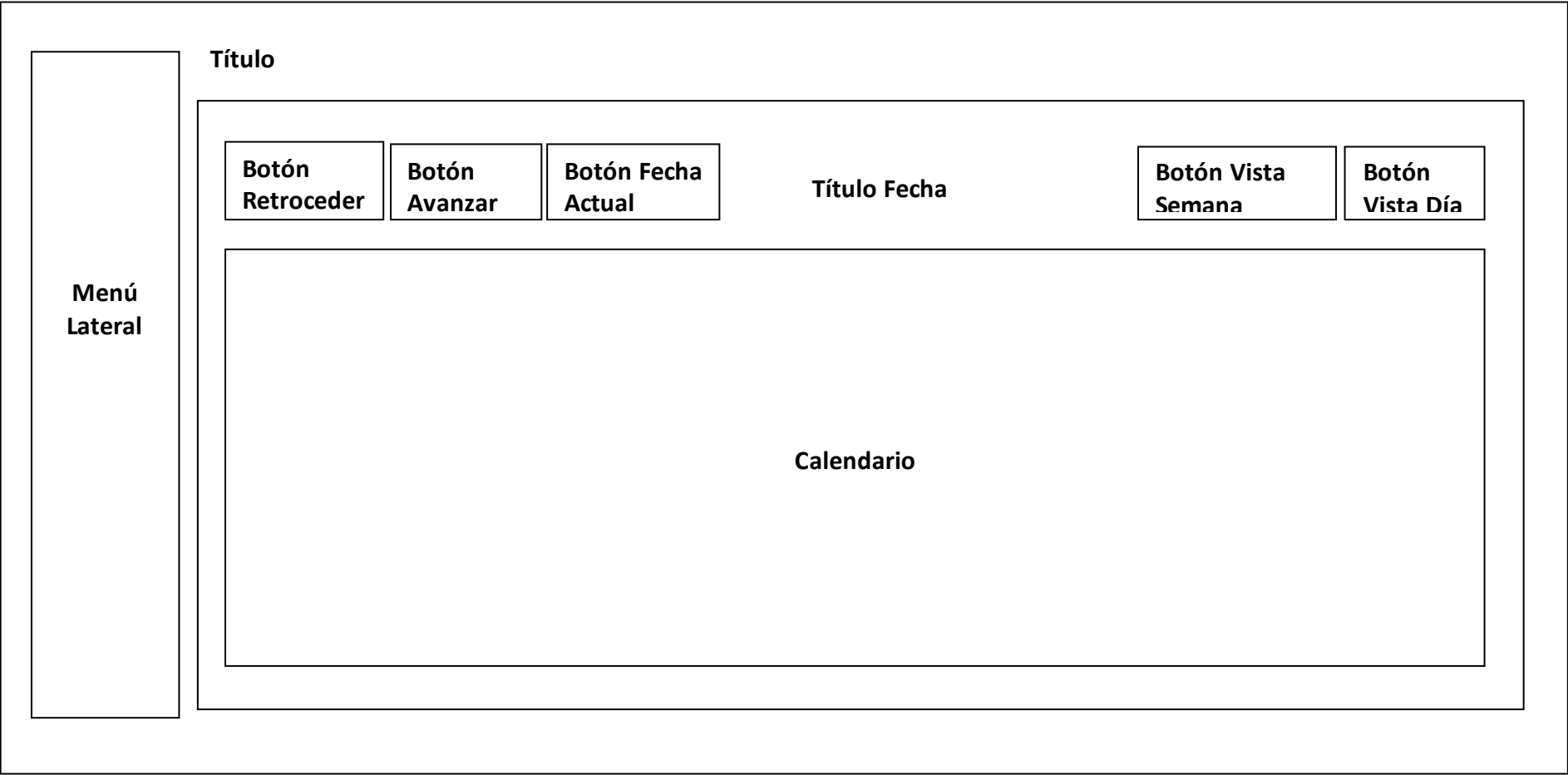


Ilustración 21:Diseño de Pantalla del Calendario de Clases

8.9. Pantalla de Entrenamiento Diario de Ejercicios

Menú Lateral

Título

Título Fecha

Campo Fecha

Buscador

Zona de listado de campos

Paginador

Buscador

Zona de listado de campos

Paginador

Ilustración 22:Diseño de Pantalla de Entrenamiento Diario de Ejercicios

8.10. Pantalla de Evolución de Ejercicios

Menú Lateral

Título

Campo fecha inicial

Campo fecha final

Selector de Socios

Buscador

Zona de listado de campos

Paginador

Ilustración 23: Diseño de Pantalla de Evolución de Ejercicios

8.11. Pantalla de Gráfica de Evolución de Ejercicios



Ilustración 24: Diseño de Pantalla de Gráfica de Evolución de Ejercicios

9. Implementación

Ahora detallaremos la estructura de nuestro proyecto a nivel de desarrollo, explicando la composición y el funcionamiento de los distintos módulos de la arquitectura. Como hemos mencionado en el punto anterior, para el desarrollo de nuestro software hemos empleado Visual Studio Code como IDE. Las capturas que adjuntaremos en el documento son dentro del propio entorno.

Esta sería la estructura de nuestro proyecto sin abrir ninguna de las carpetas iniciales.

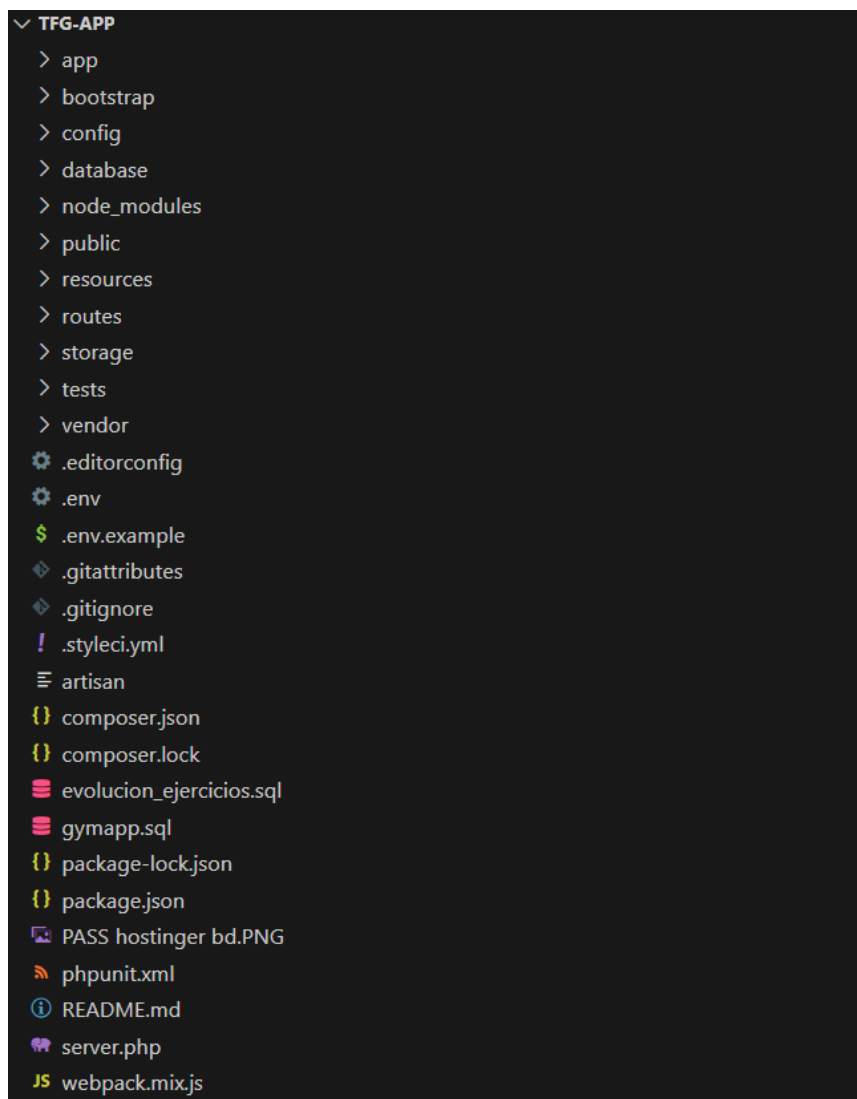


Ilustración 25: Estructura del Proyecto

9.1. Front-end

Con respecto a la parte del front-end, tenemos las carpetas llamadas **'public'** y **'resources'**.

En primer lugar, tenemos la carpeta **'public'**, la cual contiene varias subcarpetas, las cuales son las siguientes:

- **Css**
- **Js**
- **Images**
- **Font**
- **Vendor**

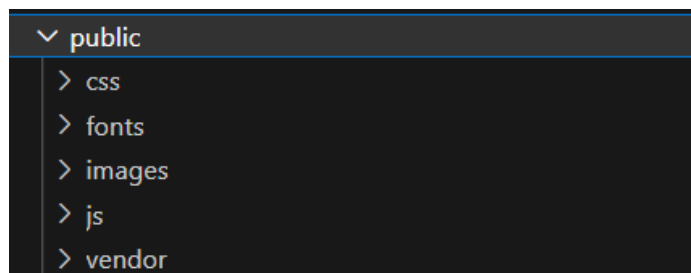


Ilustración 26: Estructura de la carpeta **'public'**

Todas estas subcarpetas contienen recursos los cuales se ejecutan en la parte del cliente que están unidos a la parte visual de la aplicación y de la programación que se ejecuta por la parte del cliente.

Por último, tenemos la carpeta **'resources'**, la cual contiene varias subcarpetas las cuales son las siguientes:

- **Images**
- **Js**
- **Lang**
- **Sass**
- **Views**



Ilustración 27: Estructura de la carpeta 'resources'

Todas estas subcarpetas contienen recursos que necesitan una compilación previa para hacer uso en la carpeta 'public' previamente mencionada, mediante **php artisan**.

En la carpeta '**views**', se encuentran todas las vistas distribuidas en diferentes subcarpetas, las cuales se visualizarán a continuación:



Ilustración 28: Estructura de la carpeta 'views'

Dentro de la carpeta '**layouts**', se encuentran las vistas relacionadas a la plantilla general de la aplicación, además de la plantilla del menú lateral, el cual se utilizará para la navegación de las distintas páginas de la aplicación.

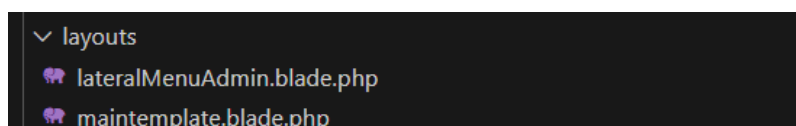


Ilustración 29: Estructura de la carpeta 'layouts'

En la carpeta **'admin'**, se encuentran todas las vistas correspondientes a la aplicación.

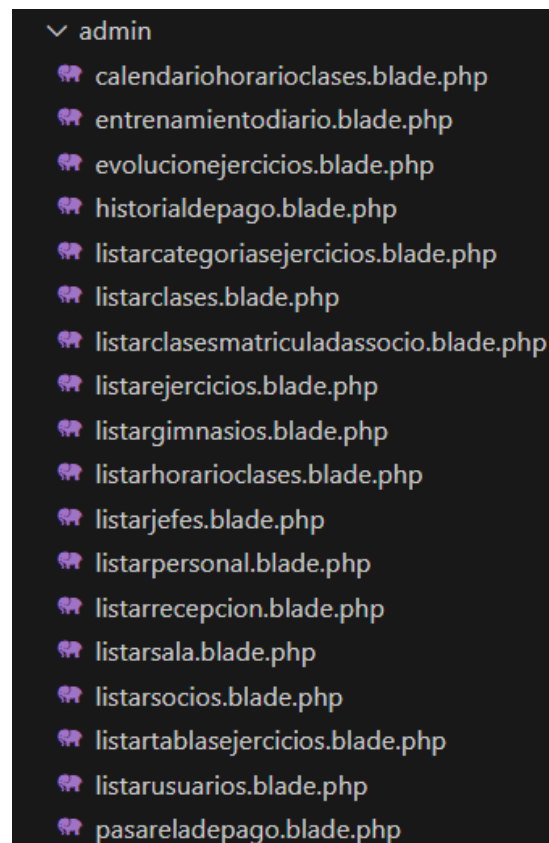


Ilustración 30: Estructura de la carpeta 'admin'

Por último, en la carpeta **'auth'**, se encuentran todas las vistas relacionadas con el login y la recuperación de contraseña.



Ilustración 31: Estructura de la carpeta 'auth'

Para la parte visual de las views donde se utilizan tablas hacemos uso de una librería denominada **'Datatable.js'** cuyo enlace es el siguiente:
<https://datatables.net/>

Un ejemplo para crear un datatable sería lo que se visualiza en la siguiente imagen:

```
var datatable = $('#table_id').DataTable({
  language: {
    "url": "//cdn.datatables.net/plug-ins/1.10.15/i18n/Spanish.json"
  },
  ajax: {
    url: "{{route('selectdataSala')}}",
    type: 'post',
    data: {
      "_token": $("meta[name='csrf-token']").attr("content")
    },
  },
  responsive: true,
  columns: [{ data: "nombre" }, { data: "capacidad" }, { data: "nombregimnasio_localidad" }],
```

Ilustración 32: Ejemplo de la visualización y creación de una datatable

La creación de un datatable (visualización de datos en una tabla) está formada por varios atributos entre ellos se encuentran los siguientes:

- **id del datatable:** hace referencia al id de un tag table.
- **language:** idioma del datatable.
- **url:** ruta a la que se dirigirá el datatable.
- **type:** depende de la ruta, será get o post.
- **_token:** es el token de seguridad.
- **responsive:** para indicar si se visualizará de forma responsive o no.
- **columns:** hace referencia a los datos de la columna que devolverá la url.

Cuando se hace la inserción, actualización y/o borrado, se utiliza Ajax para evitar hacer recargas de la página actual utilizando JQuery. Un ejemplo es el que se muestra a continuación:


```

data = datatable.row(this).data();
datatable.row(this).index

$.ajax({
  url: "{{route('getEditarDataSala')}}",
  type: "POST",
  cache: false,
  data:{
    _token:'{{ csrf_token() }}',
    id: data["id"]
  },
  success: function(dataResult){
    console.log(dataResult)
    dataJson=JSON.parse(dataResult)
    document.getElementById('nombreEditar').value=dataJson[0]["nombre"];
    document.getElementById('capacidadEditar').value=dataJson[0]["capacidad"];
    $("#id_gimnasio_selected_update").val(dataJson[0]["gimnasio_id"])

    $('#updateModal').modal('show');
  },
  error: function(e){
    console.log(e)
  }
}

```

Ilustración 33: Ejemplo de una actualización de datos de un datatable

La inserción, actualización y borrado de los datos están formada por varios atributos entre ellos se encuentran los siguientes:

- **url:** ruta a la que se dirigirá el datatable.
- **type:** depende de la ruta, será get o post.
- **cache:** si es false, obligará a que el navegador no almacene en caché las páginas solicitadas
- **data:** son los datos del request que son enviados al controller.
- **success:** cuando el resultado es satisfactorio, se realiza lo que se encuentre dentro de la función.
- **error:** cuando se produce un error, se muestra un mensaje indicándolo.

9.2. Back-end

Con respecto a la parte del back-end, tenemos las carpetas llamadas **'app'** y **'database'**.

En primer lugar, tenemos la carpeta **'app'**, la cual contiene varias subcarpetas, y en una de ellas se encuentran la subcarpeta **'Controllers'**. Los Controllers son clases que se utilizan para manejar la lógica de la aplicación y responder a las solicitudes del usuario. Un controlador actúa como intermediario entre las rutas de la aplicación y las respuestas que se deben enviar al usuario.

Los controladores en Laravel siguen el patrón de diseño Modelo-Vista-Controlador (MVC), donde el controlador se encarga de recibir las solicitudes del usuario, interactuar con los modelos y manipular los datos necesarios, y luego enviar una respuesta apropiada a la vista o a la API.

Para crear un controlador en Laravel, puedes utilizar el comando de Artisan llamado **make:controller**. Por ejemplo, para crear un controlador llamado UserController, ejecutarías el siguiente comando en la terminal:

php artisan make:controller UserController

En la carpeta **'Controllers'** se encuentran todos los controladores necesarios para este proyecto, los cuales se hará una breve explicación de cada uno de ellos.

- **CalendarioHorarioClasesController:** se encuentra las funciones correspondientes donde se visualizan las clases con sus respectivos horarios de un gimnasio en un calendario, donde al seleccionar una de ella tienes la opción de inscribirte o desinscribirte.
- **CategoriasEjerciciosController:** se encuentran las funciones correspondientes para la realización del CRUD de las categorías de los ejercicios.
- **ClasesController:** se encuentran las funciones correspondientes para la realización del CRUD de las clases de los gimnasios.
- **ClasesMatriculadasController:** se encuentran las funciones correspondientes para la visualización de las clases donde un socio se ha inscrito.

- **EjerciciosController:** se encuentran las funciones correspondientes para la realización del CRUD de los ejercicios.
- **EvolucionEjercicioController:** se encuentran las funciones correspondientes donde se visualizan dos tablas, en la primera de ellas se muestran las series, repeticiones y/o distancias objetivo de la tabla de ejercicios previamente seleccionada, y en la segunda tabla es donde se realiza la inserción, actualización y lectura de las series, repeticiones y/o distancias reales con sus pesos.
- **GimnasioController:** se encuentran las funciones correspondientes para la realización del CRUD de los gimnasios.
- **HistorialdePagoController:** se encuentran las funciones correspondientes para visualizar el listado de los pagos realizados de todos los socios.
- **HorarioClasesController:** se encuentran las funciones correspondientes de la realización del CRUD de los horarios de las clases existentes en un gimnasio.
- **JefesController:** se encuentran las funciones correspondientes para la realización del CRUD de los jefes.
- **PasareladePagoController:** se encuentran las funciones correspondientes para la realización del pago de la cuota mensual del gimnasio.
- **PersonalController:** se encuentran las funciones correspondientes para la realización del CRUD de los monitores.
- **RecepcionController:** se encuentran las funciones correspondientes para la realización del CRUD de los recepcionistas.
- **SalaController:** se encuentran las funciones correspondientes para la realización del CRUD de las salas de los gimnasios.

- **SociosController:** se encuentran las funciones correspondientes para la realización del CRUD de los socios.
- **TablasEjerciciosController:** se encuentran las funciones correspondientes para la realización del CRUD de las tablas de los ejercicios.
- **UsuariosController:** se encuentran las funciones correspondientes para la realización del CRUD de los usuarios.

Dentro de la carpeta 'Controllers', se haya la subcarpeta '**Auth**', donde se encuentran los siguientes controllers:

- **LoginController:** se encuentran todas las funciones correspondientes al inicio de sesión y al cierre de sesión de un usuario en la aplicación.
- **ResetPasswordController:** se encuentran todas las funciones correspondientes para la recuperación de las contraseñas.

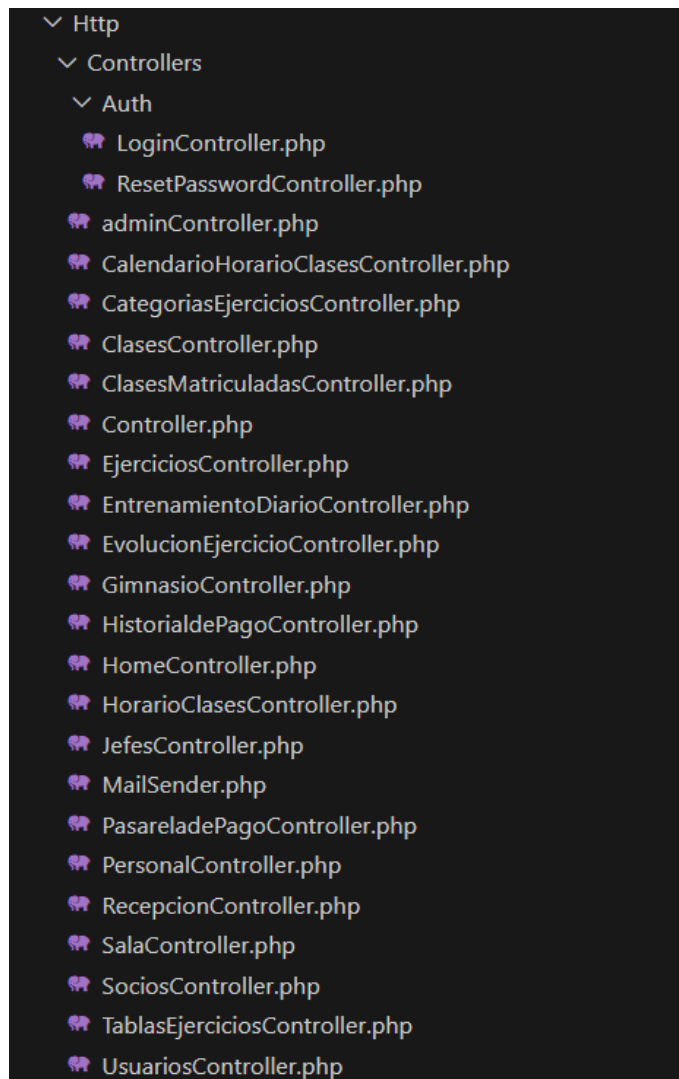


Ilustración 34: Estructura de los Controllers

Por último, tenemos la carpeta 'database', la cual contiene varias subcarpetas y una de ellas es la carpeta denominada 'migrations'. Las "migraciones" son una forma conveniente y controlada de administrar los cambios en la estructura de la base de datos. Las migraciones te permiten definir y modificar la estructura de la base de datos de manera programática, utilizando código en lugar de realizar cambios manuales en la base de datos.

Las migraciones en Laravel se crean utilizando el sistema de migraciones proporcionado por el framework. Cada migración es una clase PHP que define un conjunto de instrucciones para crear, modificar o eliminar tablas, columnas, índices u otros elementos de la base de datos.

Para crear una migración en Laravel, puedes utilizar el comando de Artisan **make:migration**. Por ejemplo, para crear una migración llamada `create_users_table`, ejecutarías el siguiente comando en la terminal:

```
php artisan make:migration create_users_table
```

Esto generará un archivo de migración en la carpeta **database/migrations**, donde puedes definir las instrucciones necesarias para crear la tabla de usuarios.

Una vez que hayas definido tus migraciones, puedes ejecutar el comando **migrate** para aplicar todas las migraciones pendientes:

```
php artisan migrate
```

Para realizar la migración de una sola tabla, se ejecuta el siguiente comando:

```
php artisan migrate --path= directorio_donde_se_encuentre_la_migración.php
```

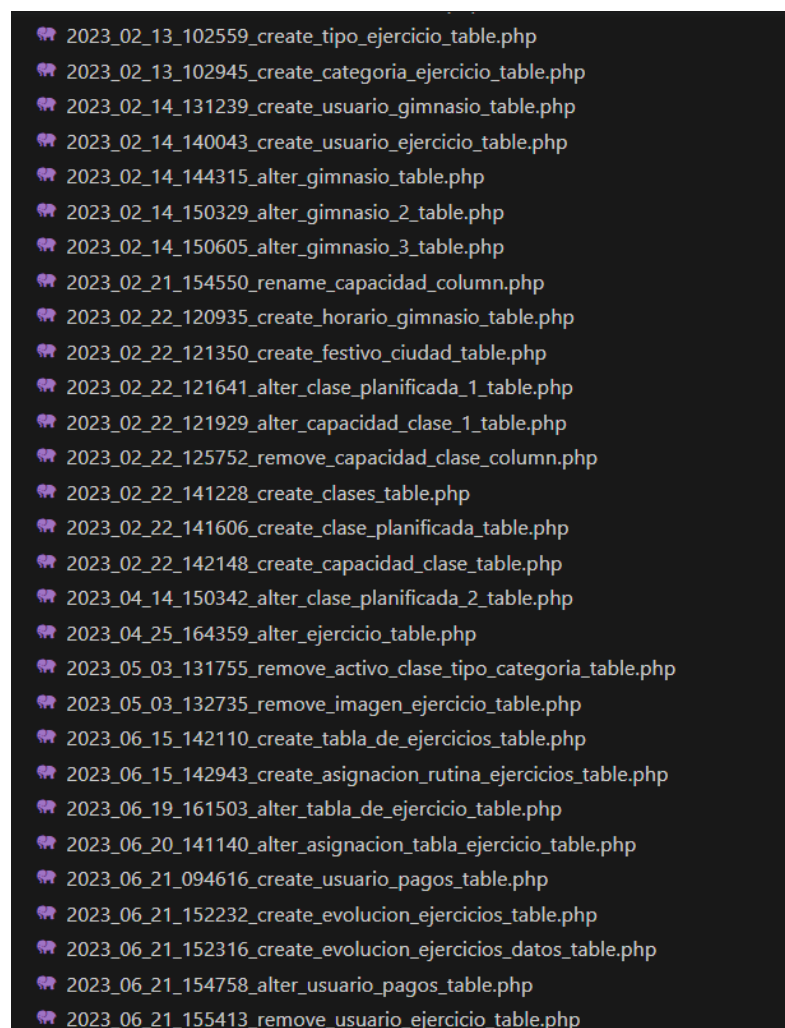


Ilustración 35: Captura de las migraciones

9.3. Routes

Con respecto a la parte de las rutas, se divide en 2 tipos:

- **Auth::routes** -> las cuales generan las rutas referentes al login de forma automática.
- **Route::group** -> es una agrupación de rutas a las que se accede si previamente se encuentra el usuario logueado. En caso contrario, se redirigirá a la ruta del login para poder iniciar sesión. A continuación, se muestra un ejemplo de ello:

```
Route::get('/listarJefes', 'JefesController@admin')->name('jefes');
Route::post('/selectdatajefes', 'JefesController@selectdatajefes')->name('selectdatajefes');
Route::post('/insertdatajefes', 'JefesController@insertdatajefes')->name('insertdatajefes');
Route::post('/deletedatajefes', 'JefesController@deletedatajefes')->name('deletedatajefes');
Route::post('/updatedatajefes', 'JefesController@updatedatajefes')->name('updatedatajefes');
Route::post('/getEditarDatajefes', 'JefesController@getEditarDatajefes')->name('getEditarDatajefes');
```

Ilustración 36: Captura de un listado de rutas

Las rutas con el método **get**, son para la llamada a la función donde se devuelve la vista.

Las rutas con el método **post** son para la inserción, actualización, borrado y listado con el objetivo de dar más seguridad a la base de datos, ya que para un método post se necesita un **token CSRF**.

Un token CSRF es un mecanismo de seguridad utilizado en aplicaciones web para prevenir ataques CSRF.

El CSRF es un tipo de ataque en el cual un sitio web malintencionado engaña al navegador de un usuario para que realice acciones no deseadas en otro sitio web en el que el usuario está autenticado. Esto se logra aprovechando el hecho de que los navegadores envían automáticamente las cookies de autenticación al hacer solicitudes a un sitio web, incluidas las solicitudes generadas por un sitio web malicioso.

Para prevenir estos ataques, Laravel (y otros frameworks y bibliotecas web) utilizan tokens CSRF. Un token CSRF es un valor único y aleatorio que se genera para cada sesión del usuario y se incluye en los formularios o en las solicitudes AJAX en la aplicación web.

10. Planificación

A continuación, se expone el modelo que hemos usado para el proceso completo de desarrollo de nuestra aplicación.

Para desarrollar nuestro proyecto, tenemos la necesidad de un periodo de aprendizaje en algunas de las tecnologías que vamos a utilizar, entonces por ello vamos a buscar reducir el riesgo en las fases más críticas, para así poder completar el sistema dentro de los plazos previstos y sin que haya ningún tipo de problema. Es por esta razón que el ciclo de vida que hemos escogido es el modelo de cascada con reducción de riesgos.

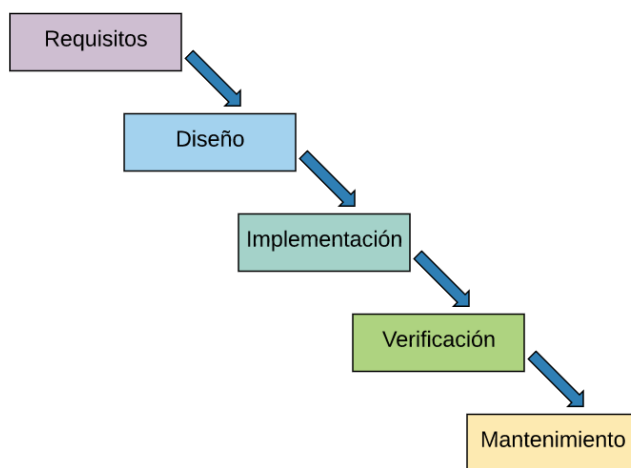


Ilustración 37: Modelo de cascada con reducción de riesgos

A continuación, vamos a hacer una explicación breve sobre cada una de estas etapas que podemos observar en la imagen.

- **Inicio:** No está presente en la ilustración, pero lo que hacemos en esta etapa es definir de manera genérica la idea que queremos llevar a cabo y se estudia la viabilidad de esta, con el fin de satisfacer las necesidades de un sector concreto, como en nuestro caso es el de los usuarios de los diferentes roles de los gimnasios. La estimación de tiempo que dedicaremos a este apartado son unas 24 horas.
- **Análisis de requisitos:** Aquí vamos a analizar más profundamente las tecnologías, para decidir las que son más acordes tanto para cubrir las necesidades del proyecto como las necesidades del desarrollador. También aquí se elabora un proyecto preliminar, que nos va a servir para ver si estas

tecnologías que hemos escogido son las correctas. La estimación de tiempo que dedicaremos a este apartado son unas 350 horas ya que, es fundamental no fallar y tener lo más completo posible este análisis para luego ahorrarnos tiempo en otras etapas.

- **Diseño del software:** Según los requisitos que hemos obtenido, procedemos a la elaboración del diseño del proyecto para, una vez finalizado, pasar a la fase de codificación. La estimación de tiempo que dedicaremos a este apartado son unas 200 horas.
- **Implementación o codificación:** En esta fase, ya empezamos con la implementación de la base de datos y el proyecto, revisando el diseño en todo momento. La estimación de tiempo que dedicaremos a este apartado son unas 400 horas porque por muy bien que hayamos realizado las etapas anteriores, suelen salir fallos y errores que nos van a restar bastante tiempo.
- **Pruebas:** Realizaremos diferentes pruebas y tests para cada apartado de la aplicación, comprobando que en ningún momento hay algún requisito que no hayamos contemplado. La estimación de tiempo que dedicaremos a este apartado son unas 130 horas.
- **Mantenimiento:** Una vez finalizado todo el proyecto, mantendremos un mantenimiento constante de la aplicación.

11. Análisis económico del proyecto

El siguiente estudio económico trata de determinar la cantidad de recursos que se han invertido en la realización de este proyecto.

El presupuesto se va a dividir en tres partes, costes materiales, costes técnicos y recursos humanos. Después de estos apartados, veremos el presupuesto para el cliente y los beneficios que podríamos sacar.

11.1. Costes materiales

Estos costes hacen referencia a las herramientas necesarias para el desarrollo del proyecto.

A continuación, mostraremos el presupuesto estimado del equipo que se ha utilizado.

Unidades	Concepto	Precio	Total
1	Ordenador portátil Características: - Procesador Intel Core i7-8750H de 6 núcleos a 2.20 GHz - Memoria RAM 8GB DDR4 - SSD NVMe M.2 de 128 GB - HDD 5.400 RPM de 1 TB - Tarjeta gráfica NVIDIA GeForce GTX 1060 6 GB GDDR5 - Resolución pantalla 1080p Full HD Porcentaje de uso 11,50%	950€	110€
		TOTAL	110€

Tabla 22: Tabla de costes materiales

11.2. Costes técnicos

Los costes técnicos hacen referencia a las herramientas, programas, librerías, sistema operativo utilizados en el proyecto.

A continuación, se muestra el presupuesto estimado del software utilizado para este desarrollo.

Unidades	Concepto	Precio	Total
1	Sistema Operativo Windows 10 Profesional 64bits – 30%	259€	77,70€
1	Microsoft Office 2020 – 15%	72€	10,80€
1	Editor de texto Visual Studio Code	0€	0€
1	Librerías y Frameworks	0€	0€
1	Servidor Hostinger	0€	43,41€
		TOTAL	131,91€

Tabla 23: Tabla de costes técnico

11.3. Costes de recursos humanos

Estos costes son los que hacen referencia a la cantidad de tiempo que hemos invertido a la hora de desarrollar el proyecto.

Podríamos dividir las horas dedicadas en dos tipos diferentes, las que realiza un desarrollador junior y las que realiza un analista, ya que son los dos roles necesarios para la realización de nuestra página web, pero vamos a ponerlo en conjunto, ya que la labor que ha hecho realmente falta es la de un ingeniero informático. Vamos a valorar el coste de ejecución de este sobre las horas dedicadas a la realización.

En el análisis y el desarrollo del software se han invertido un total de seis meses, dedicando 6 horas de trabajo diario, lo que hace un total de 1080 horas de trabajo.

Además, tenemos que añadir los 30 días de la realización de la documentación del proyecto dedicando unas 4 horas diarias, que van a sumar otras 120 horas al coste.

Los honorarios estándar para un ingeniero informático tienen un coste de 18 euros la hora.

Unidades	Concepto	Precio	Total
1080	Honorario ingeniero informático (implementación)	18€	19.440€
120	Honorario ingeniero informático (documentación)	18€	2.160€
		TOTAL	21.600€

Tabla 24: Tabla de costes de recursos humanos

11.4. Coste total del proyecto

Uniando todos los costes que hemos ido calculando y desglosando arriba, a continuación, vamos a calcular la suma total del proyecto que nos ocupa.

Concepto	Total
Costes materiales	110€
Costes técnicos	131,91€
Costes de recursos humanos	21.600€
TOTAL	21.841,91€

Tabla 25: Tabla de coste total del proyecto

12. Conclusiones

En esta última sección trataremos de explicar las conclusiones obtenidas a lo largo de todo el proyecto, así como nuevas funcionalidades que pueden ser añadidas en un futuro.

El resultado final puede consultarse en: <https://fitenergym.es/>

12.1. Conclusiones sobre el proyecto

En conclusión, en este proyecto se ha logrado desarrollar con éxito una aplicación web de gestión para una cadena de gimnasios, con el objetivo de mejorar la eficiencia operativa y proporcionar una experiencia excepcional tanto para los administradores como para los clientes.

A lo largo de este proyecto, se ha realizado un exhaustivo análisis de los sistemas de gestión existentes en la industria del fitness, lo que ha permitido identificar las áreas de mejora y las necesidades específicas de la cadena de gimnasios. Con esta información, se ha diseñado y desarrollado una aplicación web intuitiva, funcional y personalizada, que aborda estas necesidades de manera efectiva.

La aplicación web desarrollada ha demostrado ser una herramienta valiosa para los gimnasios, centralizando y automatizando tareas clave como la gestión de socios, el control de asistencia, la planificación de actividades y la gestión financiera. Esto ha contribuido a agilizar los procesos internos, reducir la carga de trabajo administrativo y mejorar la eficiencia operativa en general. Además, se realiza un seguimiento de la evolución de los ejercicios de cada socio, aportando algo que no es habitual en otras aplicaciones de gestiones de gimnasios.

Además, se ha aplicado un enfoque basado en buenas prácticas de desarrollo de software, lo que ha garantizado la calidad y fiabilidad de la aplicación. Se han realizado pruebas rigurosas para verificar su correcto funcionamiento y se ha prestado especial atención a la seguridad de los datos y la protección de la información confidencial de los usuarios.

Se espera que esta aplicación web tenga un impacto positivo en la cadena de gimnasios, al mejorar la experiencia tanto para los administradores como para los clientes. Los administradores podrán tomar decisiones más informadas y

estratégicas basadas en datos en tiempo real, mientras que los clientes disfrutarán de un acceso más fácil a los servicios y una interacción más fluida con el gimnasio.

12.2. Líneas futuras

Este proyecto también ha revelado oportunidades para futuras líneas de investigación y desarrollo. En primer lugar, se podría explorar la integración de tecnologías emergentes, como la realidad virtual o aumentada, para enriquecer la experiencia de los usuarios y fomentar la participación en el ejercicio físico. Además, la implementación de algoritmos de análisis de datos podría ofrecer información valiosa sobre las preferencias de los miembros, permitiendo una personalización aún más profunda de los servicios ofrecidos.

En términos de escalabilidad, considerar la adaptación de la aplicación a cadenas de gimnasios de diferentes tamaños y ubicaciones geográficas podría ser una dirección prometedora. Además, la seguridad cibernética y la privacidad de los datos seguirán siendo aspectos críticos a medida que la aplicación maneje información confidencial de los usuarios. Abordar estos desafíos garantizará la confianza y lealtad de los miembros a largo plazo.

En resumen, este proyecto ha logrado desarrollar una aplicación web de gestión altamente funcional y personalizada para una cadena de gimnasios, con el objetivo de mejorar la eficiencia operativa y brindar una experiencia excepcional a los usuarios. Se espera que esta solución contribuya al crecimiento y éxito continuo de la cadena de gimnasios, al tiempo que satisface las necesidades cambiantes de la industria del fitness.

13. Referencias

- MYSQL
 - <https://dev.mysql.com/doc/>
- HTML
 - <https://developer.mozilla.org/es/docs/Web/HTML>
- CSS
 - <https://developer.mozilla.org/es/docs/Web/CSS>
- Bootstrap
 - <https://getbootstrap.com/docs/4.3/getting-started/introduction/>
- AdminLTE
 - <https://adminlte.io/docs/3.2/>
- Visual Studio Code
 - <https://code.visualstudio.com/>
- Laravel
 - <https://laravel.su/docs/7.x/documentation>
- JavaScript
 - <https://www.w3schools.com/jsrEF/default.asp>
 - <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

- Stripe
 - <https://stripe.com/docs>

- Hostinger
 - <https://www.hostinger.es/>

- Chart.js
 - <https://www.chartjs.org/docs/latest/>